生物信息学综合实验

学 期 末 实 验 报 告

（十个实验全记录）

课程名称：	生物信息学综合实验
报告名称：	生物信息学综合实验报告
姓　　名：	【姓名】
学　　号：	2023214116
专业班级：	【专业班级】
指导教师：	【教师姓名】
完成日期：	2026 年 10 月 27 日

2026 年 10 月

目　　录

实验一　NCBI 数据库初识.....

实验一　NCBI 数据库初识.....1

一、实验目的.....1

二、实验内容.....2

三、实验步骤与结果.....3

3.1 查资料：NCBI 是个什么机构.....3

3.2 NCBI 主页.....3

3.3 All Resources 资源列表.....4

3.4 PubMed 文献库.....4

3.5 Nucleotide 核酸数据库.....5

3.6 Protein 蛋白质数据库.....5

3.7 SRA 测序数据存档库.....6

四、小结.....6

实验二　NCBI 基因搜索与下载.....7

一、实验目的.....7

二、实验内容.....7

三、实验步骤与结果.....8

3.1 选哪个基因：TP53.....8

3.2 在 NCBI 查找 TP53 基因.....8

3.3 查看 TP53 的基本信息.....9

3.4 下载 TP53 的 mRNA 序列（FASTA）.....9

3.5 随机挑选 5 个基因批量下载.....10

3.6 在本地检查下载结果.....11

四、小结.....11
实验三　基因功能分析（BLAST）.....12
一、实验目的.....12
二、实验内容.....12
三、实验步骤与结果.....13
3.1 BLAST 五兄弟的分工.....13
3.2 用 TP53 mRNA 在线比对 Nr 数据库.....13
3.3 在线比对结果解读.....14
3.4 本地 BLAST：建库.....15
3.5 本地 blastn 比对.....16
四、小结.....17
实验四　Linux 基本操作.....17
一、实验目的.....17
二、实验内容.....18
三、实验步骤与结果.....19
3.1 环境说明：先在本地 Ubuntu 上把命令练熟.....19
3.2 修改密码 passwd.....19
3.3 cd 改变工作目录.....20
3.4 mkdir / touch 创建目录和文件.....20
3.5 mv 移动与重命名.....21
3.6 rm 删除.....22
3.7 vi 编辑文件.....22
四、踩坑与小结.....23
实验五　Shell 编程初步.....23
一、实验目的.....24
二、实验内容.....24

三、实验步骤与结果.....24
3.1 环境变量.....25
3.2 if-else：检查 test.txt 是否存在.....25
3.3 for 循环：100 个随机数里挑大于 50 的.....26
3.4 while 循环：同样的事再来一遍.....26
3.5 awk / sed / grep 三剑客.....27
四、小结.....28
实验六　高通量测序数据下载.....28
一、实验目的.....28
二、实验内容.....29
三、实验步骤与结果.....29
3.1 SRR26342669 的基本信息.....29
3.2 浏览器下载 fastq.....30
3.3 prefetch 下载.....31
3.4 fasterq-dump 转换成 fastq.....32
3.5 Shell 脚本：fastq 转 fasta.....32
四、小结.....33
实验七　高通量测序数据格式.....33
一、实验目的.....33
二、实验内容.....34
三、实验步骤与结果.....34
3.1 fastq 格式逐行拆解.....34
3.2 Phred 质量值的计算公式.....35
3.3 Shell 程序计算 Q20/Q30.....36
四、小结.....37
实验八　基因组 Survey 分析.....37

一、实验目的.....38
二、实验内容.....38
三、实验步骤与结果.....38
3.1 数据下载与转换.....38
3.2 jellyfish 做 k-mer 计数.....39
3.3 GenomeScope 拟合分析.....40
四、小结.....42
实验九　参考基因组下载与统计.....42
一、实验目的.....42
二、实验内容.....42
三、实验步骤与结果.....43
3.1 一个需要说明的发现.....43
3.2 下载参考基因组.....43
3.3 Python 统计脚本.....44
四、小结.....45
实验十　全基因组变异位点挖掘.....45
一、实验目的.....46
二、实验内容.....46
三、实验步骤与结果.....46
3.1 数据下载与转换.....46
3.2 bwa 比对.....47
3.3 SAM/BAM 文件格式解析.....49
3.4 bcftools 变异检测.....50
3.5 VCF 格式解析与统计.....51
四、小结.....52
写在最后：十个实验的总结.....52

一、整个课程做了什么.....52
二、踩坑记录.....53
三、几点真心的体会.....53
四、给课程的反馈.....54

说明：目录由域代码生成，打开文档后请在目录上点击右键选择“更新域”来刷新页码。

实验一　NCBI 数据库初识

一、实验目的

了解 NCBI 数据库的基本结构和主要内容，熟悉几个最常用子库（PubMed、Nucleotide、Protein、SRA）的页面布局和检索方式，为后面的实验打基础。

二、实验内容

- 网络搜索 NCBI 数据库的介绍；
- 打开 NCBI 数据库网址，浏览主页；
- 了解并打开 NCBI PubMed 数据库；
- 了解并打开 NCBI Nucleotide 数据库；
- 了解并打开 NCBI Protein 数据库；
- 了解并打开 NCBI SRA 数据库。

三、实验步骤与结果

3.1 查资料：NCBI 是个什么机构

动手之前我先搜了一下 NCBI 的背景资料。NCBI 全称是 National Center for Biotechnology Information（美国国家生物技术信息中心），1988 年隶属于美国国家医学图书馆（NLM），是生信圈最常用的公共数据库之一。它维护着 GenBank 核酸序列数据库、PubMed 文献库、SRA 测序数据存档库等几十个子库，还提供 BLAST 在线比对、引物设计这些工具。我们课上学过的很多分析流程，起点几乎都是“去 NCBI 下一条数据”，所以第一个实验先把它摸熟是很有必要的。

NCBI 的官方网址是 https://www.ncbi.nlm.nih.gov/ ，直接在浏览器地址栏输入就能打开，不需要注册账号（下载和检索基本功能都是免费开放的）。

3.2 NCBI 主页

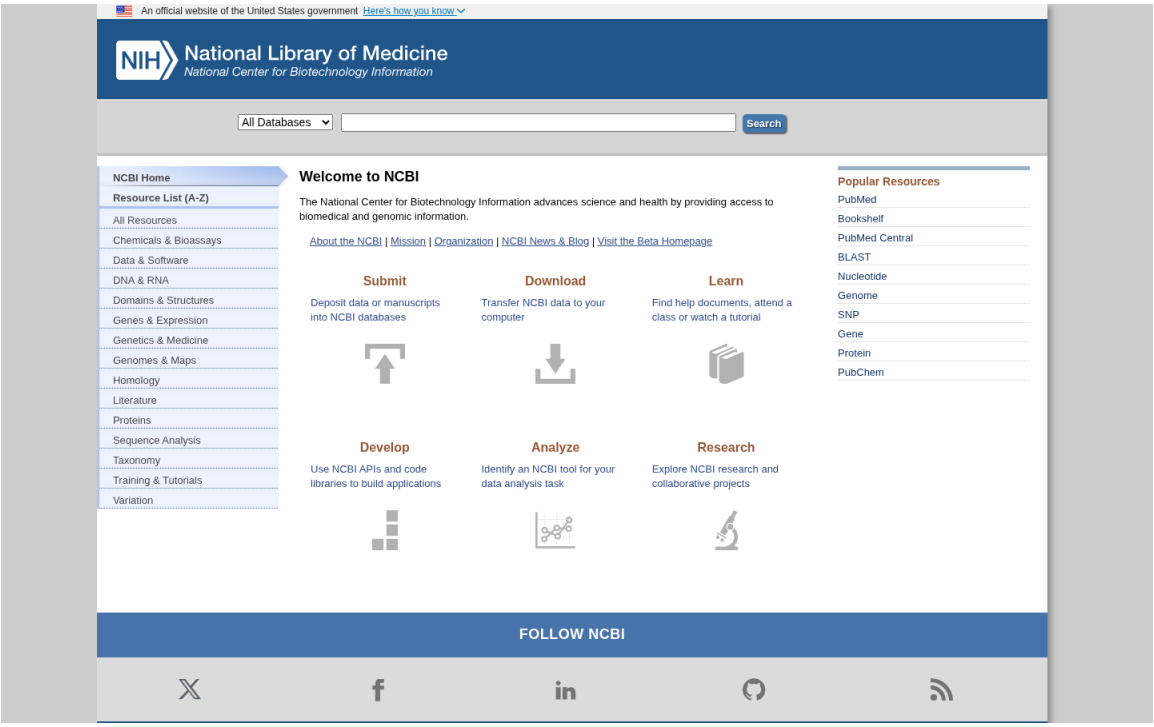


图 1-1　NCBI 数据库主页

主页顶部的输入框就是全局检索入口，下拉框可以选要检索的数据库。页面左侧是按类别归类的资源列表，右侧有 Popular Resources 一栏，列出了 PubMed、BLAST、Gene 这些高频入口。中间六个大图标分别是 Submit、Download、Learn、Develop、Analyze、Research，相当于把常用功能做了个导览。我后面几个实验基本都是从这个页面出发的。

3.3 All Resources 资源列表

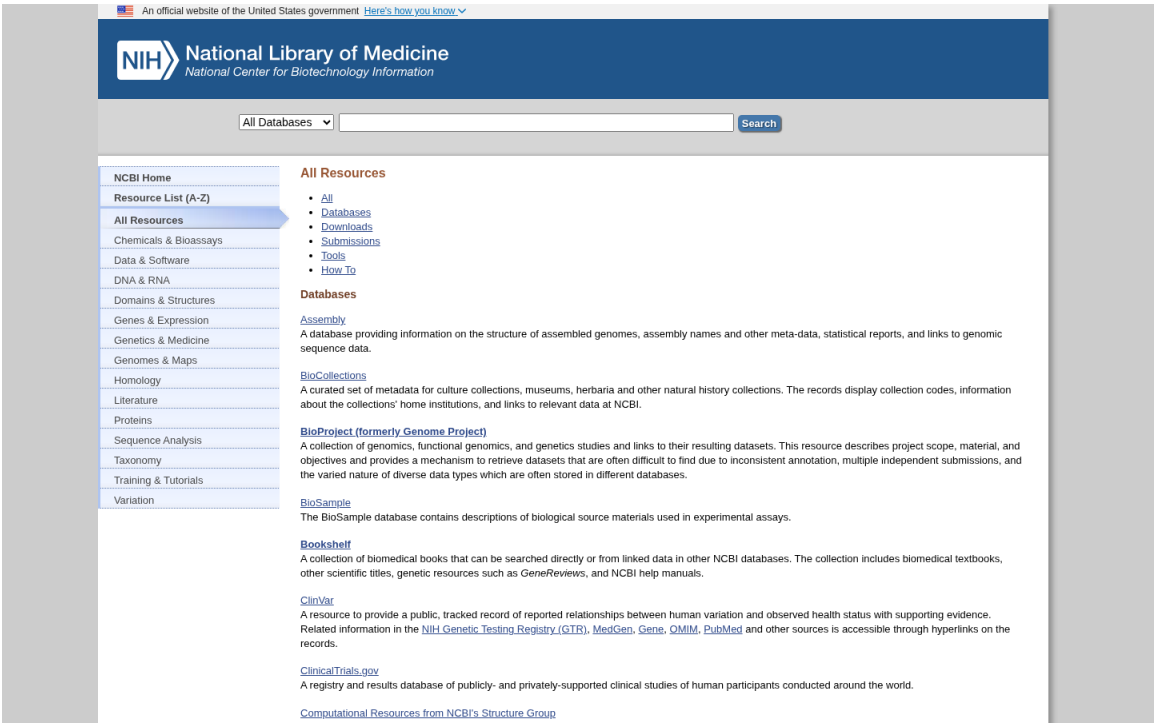


图 1-2　NCBI 全部资源（All Resources）列表页

点进 Site Guide 里的 All Resources，能看到 NCBI 全部子库的清单，按 Literature、Genes、Proteins、Genomes 等大类分组。数了一下大概有几十个库，比我想象的多得多。不过课程里主要用到的就是 PubMed、Gene、Nucleotide、Protein、SRA、Assembly 和 BLAST 这几个，下面挨个点开看看。

3.4 PubMed 文献库

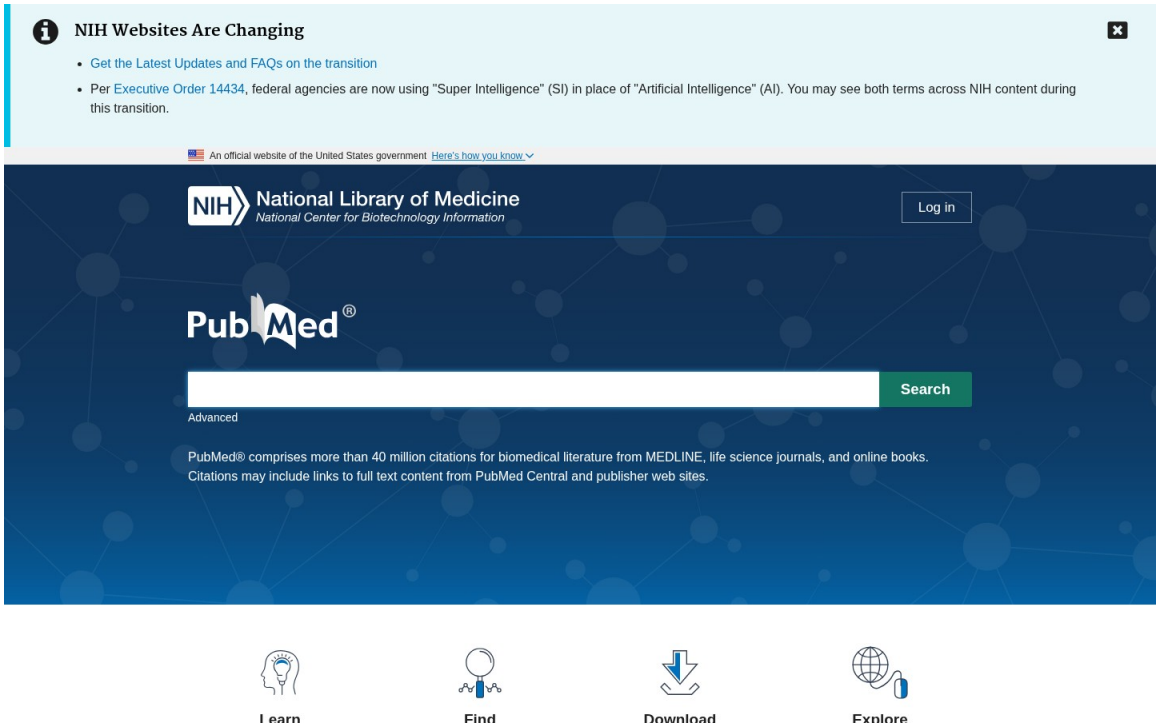


图 1-3　PubMed 数据库主页

PubMed 是 NCBI 的文献检索库，收录了三千多万篇生物医学文献的摘要（大部分来自 MEDLINE）。页面就是一个大大的搜索框，风格比主页简洁很多。

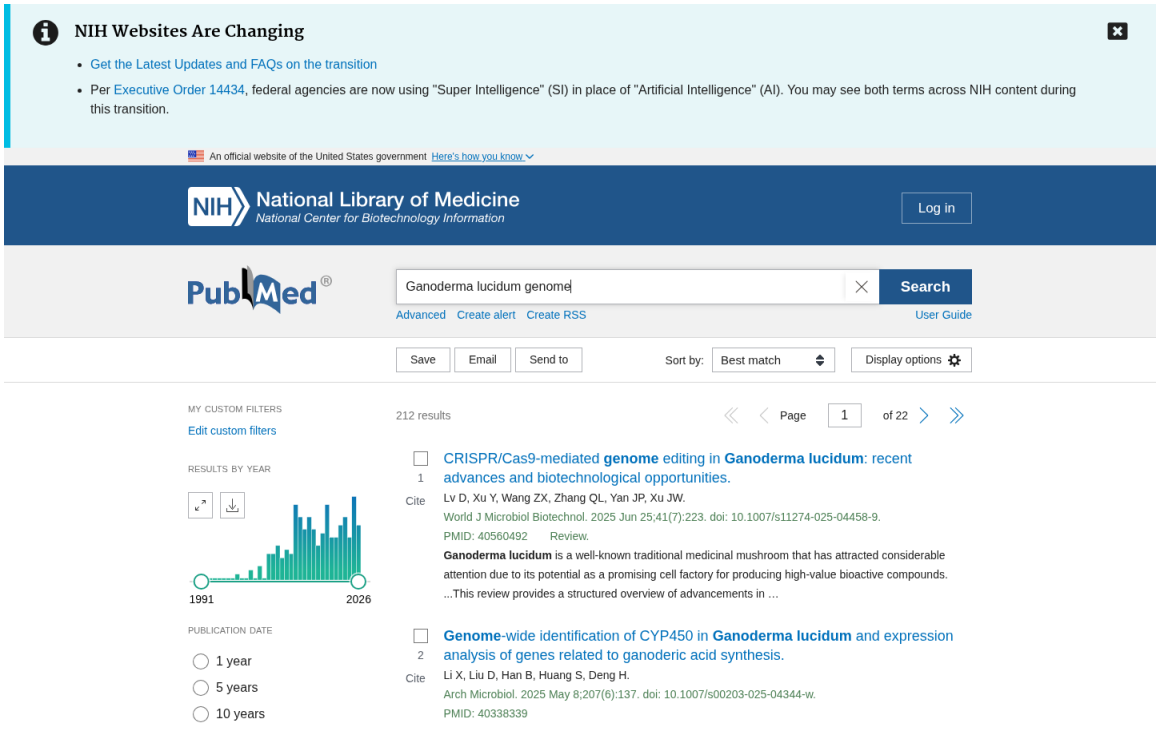


图 1-4　在 PubMed 中检索 “Ganoderma lucidum genome” 的结果

我试着搜了一下 Ganoderma lucidum genome（灵芝基因组，和后面实验八的数据对应），返回了三百多条记录。左侧可以按年份、文章类型过滤，点开每条记

录能看到摘要、作者、期刊信息，有免费全文的会标着 PMC 的链接。检索框支持字段限定，比如 Ganoderma[Title] 表示只搜标题里带这个词的，这个语法后面还挺常用。

3.5 Nucleotide 核酸数据库

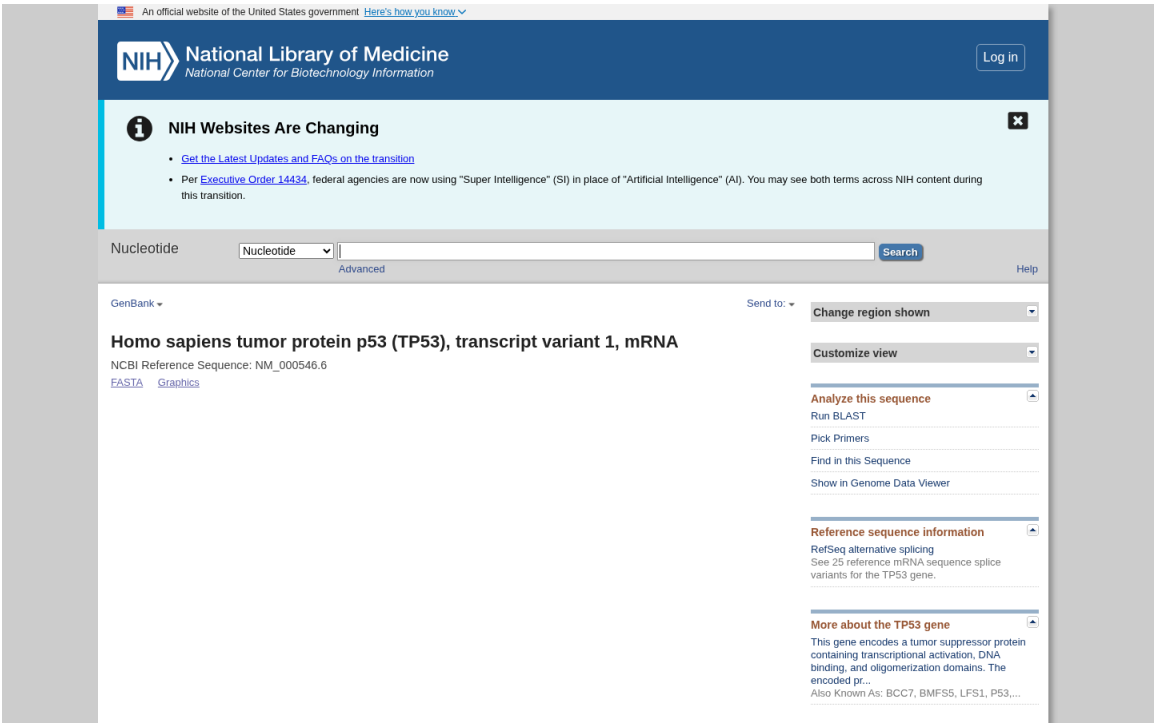


图 1-5　Nucleotide 数据库中 TP53 的 mRNA 记录（NM_000546.6）

Nucleotide 库是 GenBank 及其合作库（EMBL、DDBJ）的核酸序列汇总。我点开了一条人类 TP53 基因 mRNA 的 RefSeq 记录（收录号 NM_000546.6）。页面上方是 LOCUS 信息（序列长度 2521 bp、线性 mRNA、最后更新日期），下面依次是来源物种、参考文献、FEATURES 表格（CDS、外显子位置等注释）和最底下的 ORIGIN 原始序列。这个库是实验二下载基因序列的主要来源。

3.6 Protein 蛋白质数据库



图 1-6　Protein 数据库中 TP53 蛋白记录（NP_000537.3）

Protein 库和 Nucleotide 结构类似，只是存的是氨基酸序列。上面这条是 TP53 的翻译蛋白 NP_000537.3，一共 393 个氨基酸。同样有 FEATURES 注释，比如 DNA 结合域的位置就在这里标注。注意区分：NM 开头的是 mRNA，NP 开头的是对应的蛋白，两个编号只差一个字母。

3.7 SRA 测序数据存档库

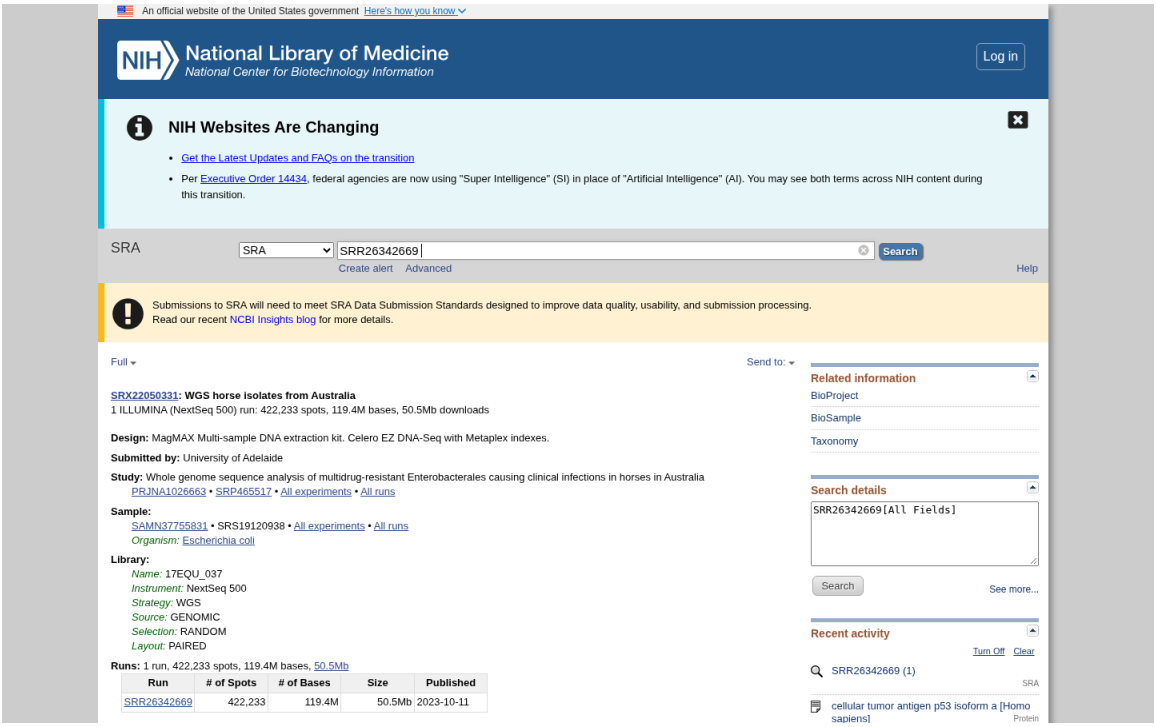


图 1-7　SRA 数据库中 SRR26342669 的检索结果

SRA（Sequence Read Archive）存放二代测序的原始测序数据。我检索了 SRR26342669（实验六要用的数据），可以看到这条 run 的基本信息：大肠杆菌 WGS 数据、Illumina NextSeq 500 平台、422,233 个 spots、约 119 Mb，属于 BioProject PRJNA1026663（澳大利亚马源分离株的研究）。SRA 的编号体系是 SRP（项目）→ SRS（样本）→ SRX（实验）→ SRR（一次测序 run），实验六、八、十的下载数据都从这个库里来。

四、小结

这个实验没有太多技术含量，主要是把 NCBI 的家底摸清楚。对我来说最有用的收获有两个：一是搞明白了各个子库的分工和编号规则（NM/NP/SRR 这些前缀不再是一堆乱码），二是发现右侧的 Popular Resources 入口比从搜索框里盲搜效率高很多。后面九个实验基本都是在这些页面上展开的，先把路认熟，走路才快。

实验二　NCBI 基因搜索与下载

一、实验目的

掌握 NCBI 的基因搜索、基因信息查看和序列查找方法，学会下载单个基因的 FASTA 序列，以及多个基因的批量下载。

二、实验内容

- NCBI 基因搜索；
- 基因信息查看；
- 基因序列 fasta 文件下载；
- 多个基因序列批量下载。

三、实验步骤与结果

3.1 选哪个基因：TP53

自选基因我选了 **TP53**（人类肿瘤蛋白 p53 基因）。选它的理由有三个：第一，p53 是最有名的肿瘤抑制基因，被称为“基因组守护者”，课上和文献里出现频率极高，功能注释非常完善，查起来不容易踩坑；第二，它的 mRNA 和蛋白序列长度适中（2521 bp / 393 aa），比对结果好解读；第三，它在不同物种里有大量同源序列，正好适合实验三做 BLAST 同源性分析。

3.2 在 NCBI 查找 TP53 基因

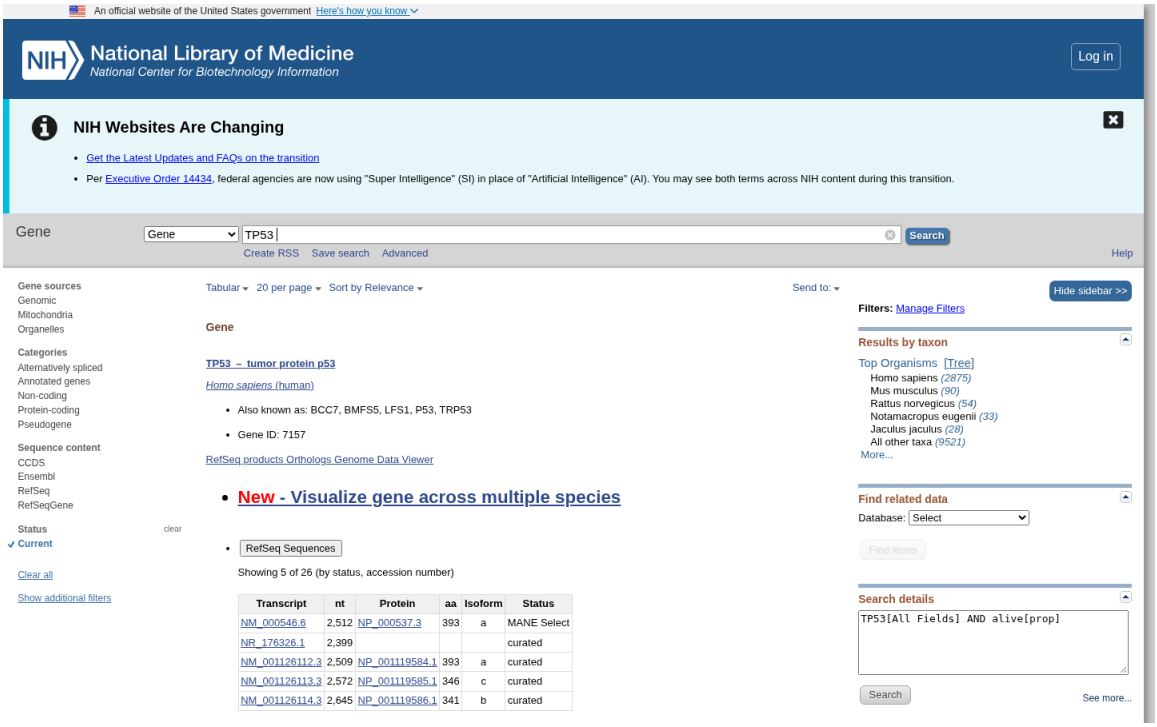


图 2-1　在 Gene 数据库中检索 TP53

在 NCBI 主页把数据库切换到 Gene，输入 TP53 回车。结果列表第一条就是目标：人类 tumor protein p53，Gene ID 7157，下面还跟着一堆其他物种的同源基因（小鼠 Trp53、斑马鱼 tp53 之类）。列表里每条记录前面的小图标是基因类型标记，蛋白编码基因是一个小圆点图标。

3.3 查看 TP53 的基本信息

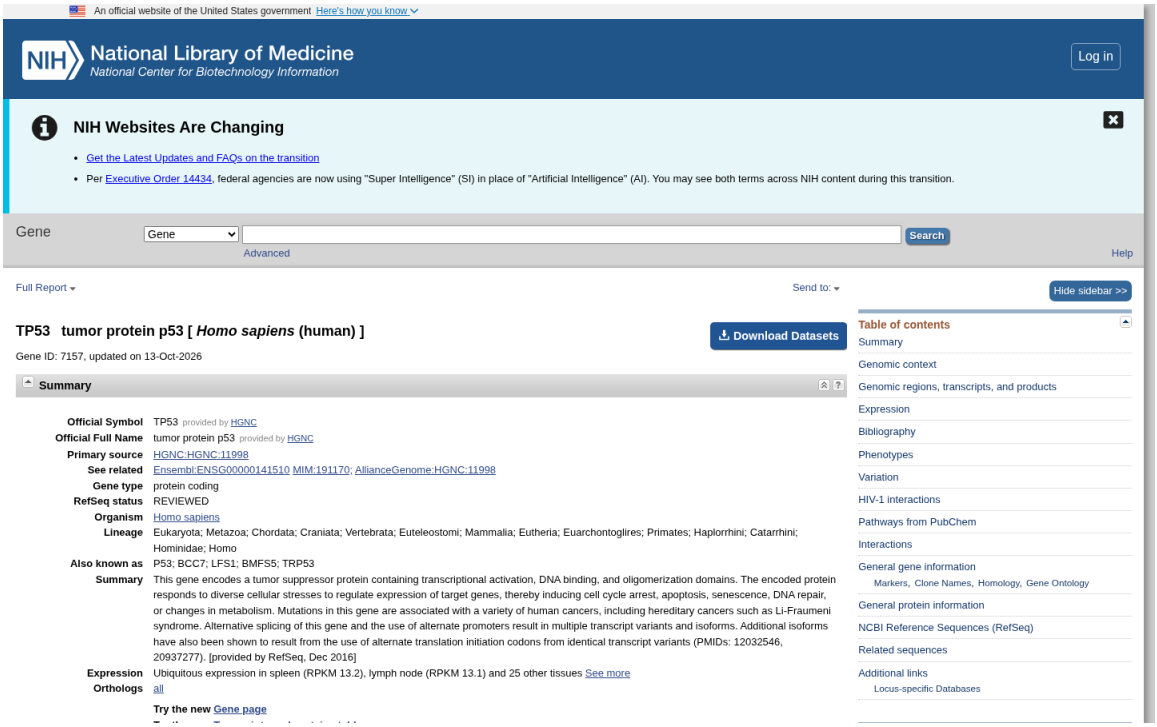


图 2-2　TP53 基因详情页（Gene ID 7157）

点进人类的 TP53，详情页内容非常丰富：顶部摘要区写明了基因功能（转录因子，响应细胞应激，调控细胞周期阻滞、凋亡等）；中间的 Expression 部分有不同组织里的表达量柱状图（它在脾脏、胸腺这些组织里表达量比较高）；图里还能看到它在 17 号染色体短臂（17p13.1）上的位置，以及基因组浏览器视图。页面往下拉有 Variation、Interactions、Pathways 等板块，数据量很大，我重点看了 Summary、Gene Ontology 和 NCBI Reference Sequences 这三块，确认转录本信息在 RefSeq 里。

3.4 下载 TP53 的 mRNA 序列（FASTA）

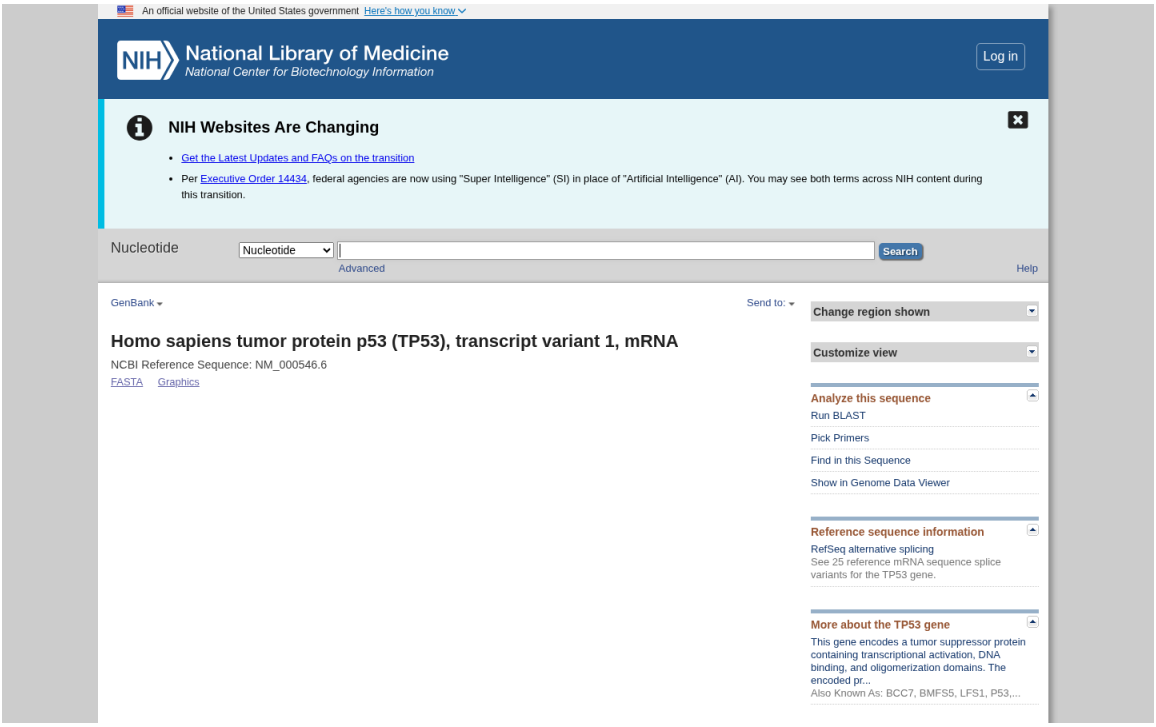


图 2-3　TP53 的 RefSeq mRNA 记录页面（Nucleotide 库）

下载序列要从 Nucleotide 库走。在基因详情页找到 NCBI Reference Sequences 里的 mRNA 条目，点 NM_000546.6 进入 Nucleotide 记录页（图 2-3）。这个页面在实验一已经见过，底部 ORIGIN 就是序列本体。

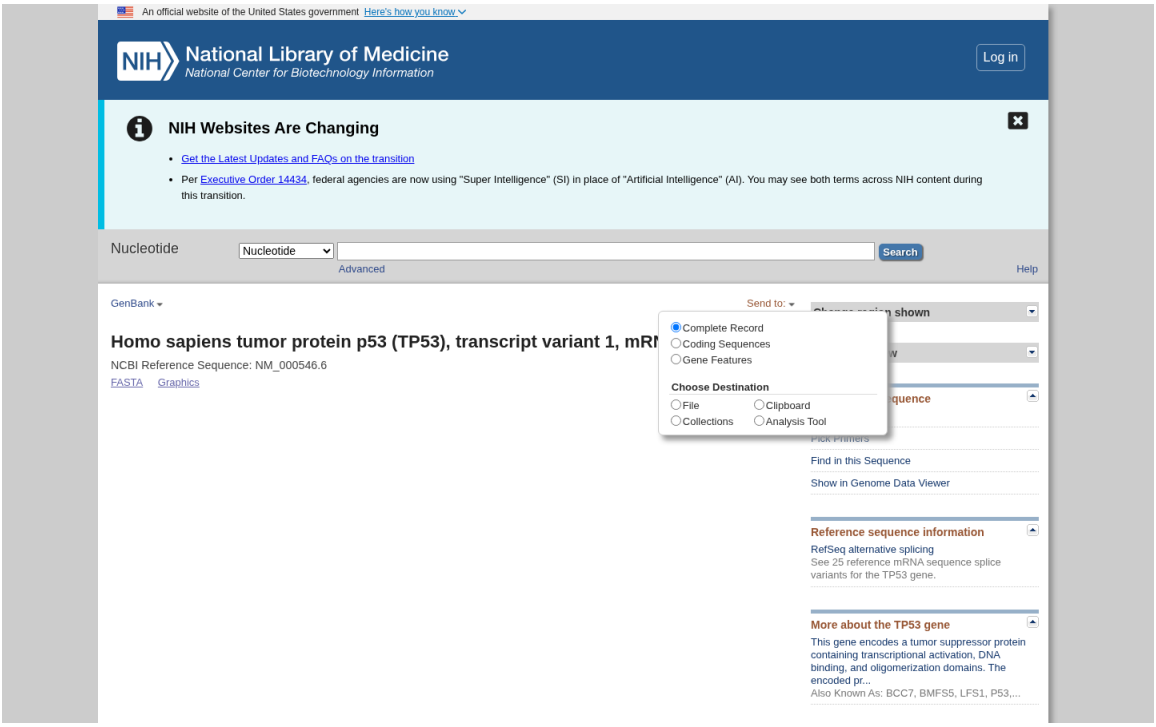


图 2-4　使用 Send to → File → FASTA 下载序列

下载的操作路径是：点页面上方的 **Send to** 按钮 → 选择目的地 **File** → 格式选 **FASTA** → 点 Create File（图 2-4 就是选好之后的对话框状态）。浏览器会把 sequence.fasta 下载下来，我把它改名为 tp53_mrna.fasta 存进 ~/bioinfo/exp2。文件开头是 “>NM_000546.6 ...” 的描述行，后面跟着 70 个字符一行的碱基序列。

3.5 随机挑选 5 个基因批量下载

批量下载这一步，我随手选了 5 个人类明星基因（其实也谈不上完全随机，都是平时文献里眼熟的）：BRCA1（乳腺癌易感基因）、TP53（本实验主角）、EGFR（表皮生长因子受体，肺癌靶向药的目标）、KRAS（RAS 家族原癌基因）和 APOE（载脂蛋白 E，阿尔茨海默病风险位点）。它们的 mRNA 收录号分别是 NM_007294.4、NM_000546.6、NM_005228.5、NM_033360.4、NM_000041.4。

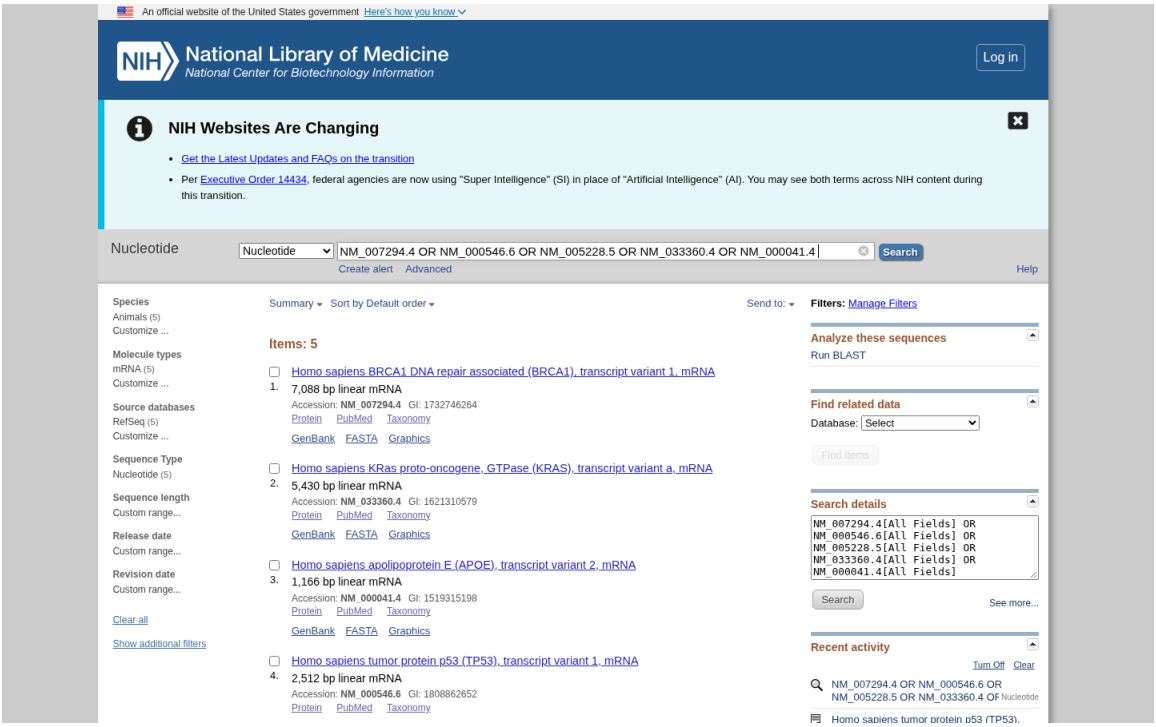


图 2-5　在 Nucleotide 库中用 OR 语法一次检索 5 个收录号

批量检索的技巧是把 5 个收录号用 OR 连起来，一次性放进 Nucleotide 的搜索框 NM_007294.4 OR NM_000546.6 OR NM_005228.5 OR NM_033360.4 OR NM_000041.4。检索结果正好 5 条记录（图 2-5），说明每个号都命中了。

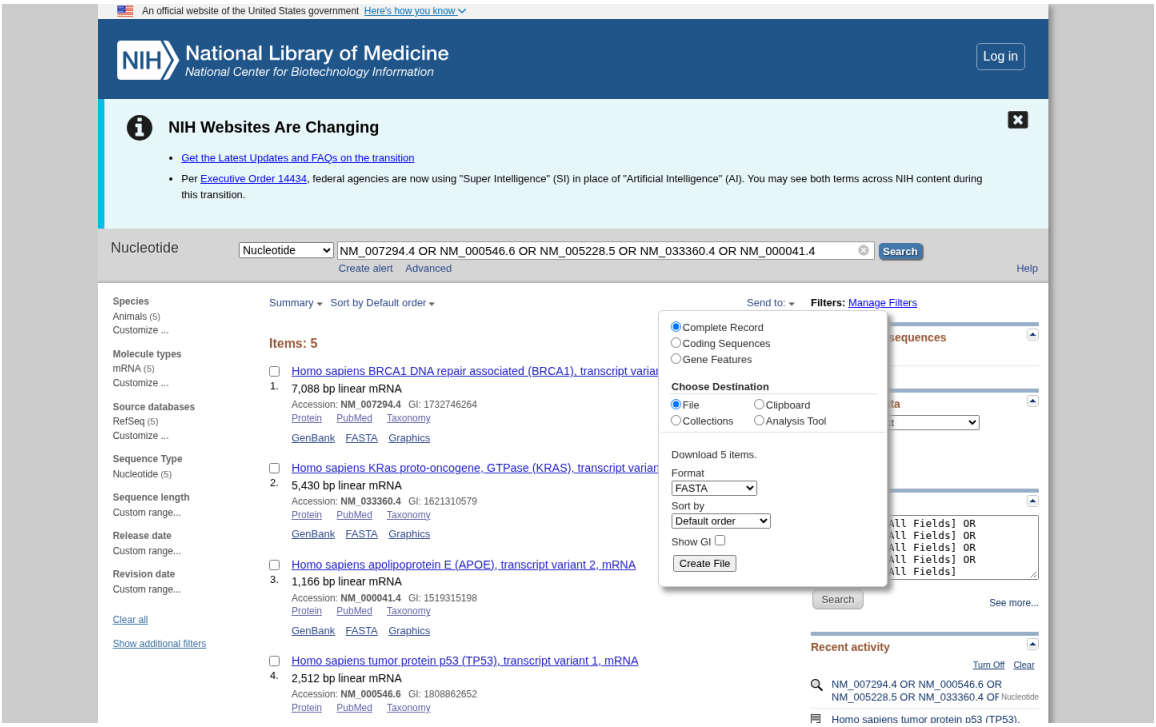


图 2-6　批量下载：Send to → File → FASTA（5 条记录一起导出）

然后还是老办法：Send to → File → Format 选 FASTA → Create File，这次导出的就是一个包含 5 条序列的 five_genes.fasta 文件（图 2-6 是操作时的对话框）。

3.6 在本地检查下载结果

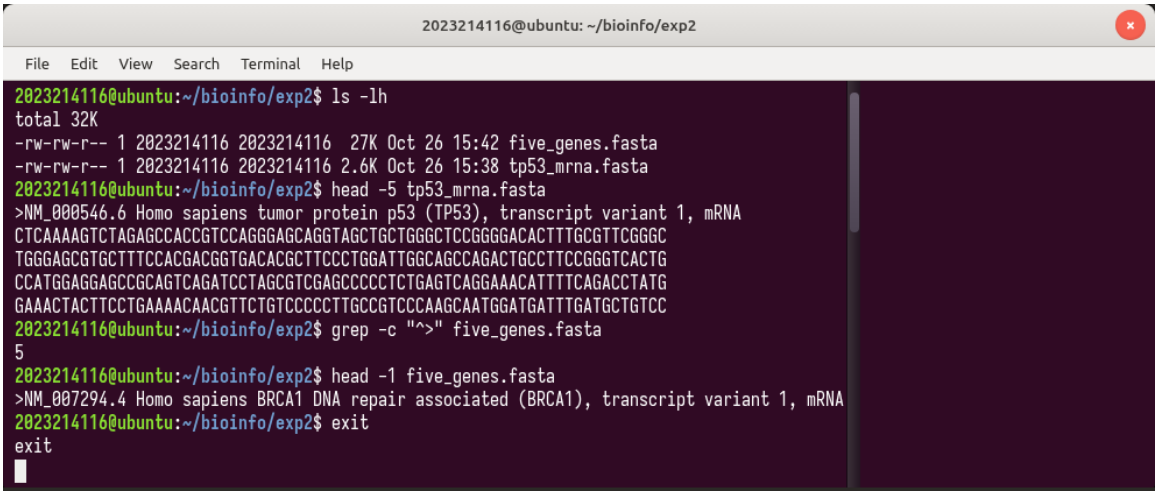


图 2-7　在本地检查下载的两个 fasta 文件

把两个文件放进 ~/bioinfo/exp2 目录，检查一下内容。用 head -5 看 tp53_mrna.fasta 的开头，用 grep -c "^>" 数 five_genes.fasta 里的序列条数，输出正好是 5（图 2-7），和下载时选的记录数一致。第一条序列的描述行也和网页上显示的对得上。这两个文件在实验三做本地 BLAST 时还要当数据库用。

四、小结

这个实验的难点不在操作，而在“知道去哪儿找”：同一个基因的信息分散在 Gene、Nucleotide、Protein 几个库里，刚上手时很容易绕晕。我的经验是：看基因功能注释去 Gene 库，拿序列去 Nucleotide 库（NM/NP 编号），两个库之间有互相跳转的链接。另外 Send to 的 File 导出支持很多格式（GenBank、FASTA、ASN.1 等），平时用得最多的是 FASTA 拿序列、GenBank 拿注释。批量下载只要掌握 OR 检索语法就非常省事，比一条条点快多了。

实验三　基因功能分析（BLAST）

一、实验目的

掌握利用 BLAST 程序进行基因功能分析的方法，包括 BLAST 程序包里各子程序的适用场景、NCBI 在线 BLAST 的使用，以及在本地 Linux 环境里做 BLAST 比对

二、实验内容

- 查询并掌握 blast 程序包中 blastn、blastp、blastx、tblastn、tblastx 的使用场景；
- 独立选择一个基因序列，选择 Nr 数据库进行同源性比对；
- 查看基因序列进行 Nr 数据库比对的结果；
- 利用服务器进行上述基因的本地化 BLAST 比对。

三、实验步骤与结果

3.1 BLAST 五兄弟的分工

先查资料把五个子程序的区别搞清楚。BLAST 的核心思想是把查询序列和数据库序列做局部比对，五个子程序的差别在于“查询序列是什么类型”和“数据库是什么类型”，中间可以经过翻译。我整理成了下面这张表：

表 3-1　BLAST 五个子程序的适用场景

程序	查询序列	数据库	典型使用场景
blastn	核酸	核酸	引物/基因在基因组上定位，物种鉴定（如 16S）
blastp	蛋白	蛋白	未知蛋白的功能注释，找同源蛋白
blastx	核酸（翻译后）	蛋白	新转录本/CDS 的功能预测，找远源蛋白
tblastn	蛋白	核酸（翻译后）	在基因组/转录组里找某个蛋白的编码区
tblastx	核酸（翻译后）	核酸（翻译后）	两条核酸的编码区相似性比对，最耗时

记忆窍门：n 代表 nucleotide、p 代表 protein、x 代表“查询序列被翻译”，t 开头代表“数据库被翻译”。评估比对结果主要看三个指标：E 值（越小越可信，小于 1e-5 一般算显著）、一致性（Identities 百分比）和比对长度。

3.2 用 TP53 mRNA 在线比对 Nr 数据库

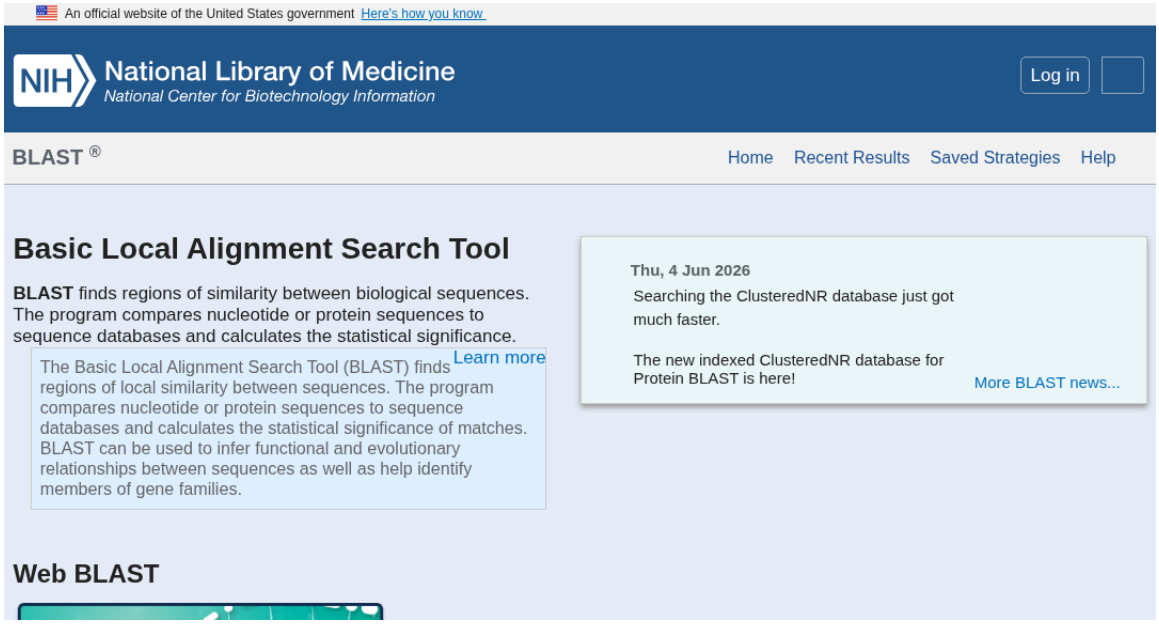


图 3-1　NCBI BLAST 主页

打开 BLAST 主页（blast.ncbi.nlm.nih.gov），五个子程序的入口一字排开。因为查询序列是 TP53 的 mRNA（核酸），要对比的 Nr 数据库里既有核酸集合（nr/nt）也有蛋白集合（nr），我先按题目要求选 Nucleotide collection (nr/nt) 做 blastn。

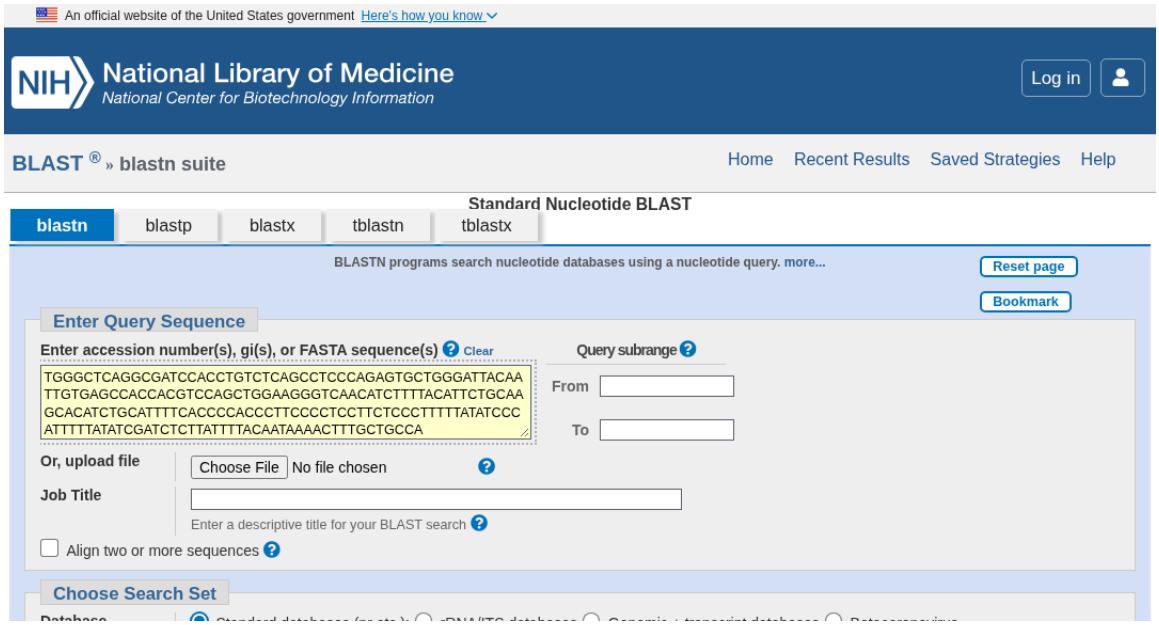


图 3-2　blastn 提交页面：序列已粘贴，数据库选 Nucleotide collection (nr/nt)

点 Nucleotide BLAST 进入提交页，把实验二下载的 TP53 mRNA 序列（FASTA 格式，连描述行一起）粘进 Enter Query Sequence 输入框；Database 选 **Nucleotide**

collection (nr/nt)，算法保持默认的 megablast（适合找高相似序列），然后点 BLAST 提交（图 3-2）。

3.3 在线比对结果解读

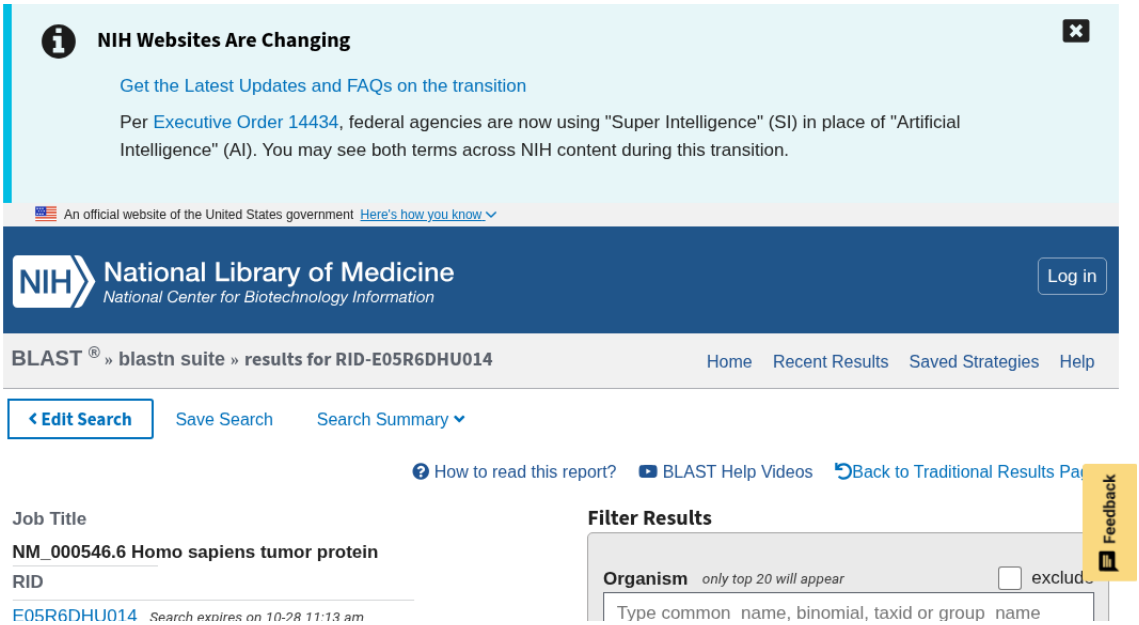


图 3-3　blastn 结果页：Descriptions 命中列表

等了大约一分钟出结果。页面从上到下分三块：Graphic Summary（彩色条带图，一条带一个命中，颜色代表比分高低）、Descriptions（命中列表，图 3-3）、Alignments（逐条比对详情）。命中列表前几页几乎被灵长类的 p53 mRNA 刷屏：人类 TP53 自己 100% 命中（Query Cover 100%、E 值 0.0），黑猩猩、恒河猴的 p53 都在 95% 以上，再往后是狗、小鼠、斑马鱼，相似度依次下降，完全符合进化的亲缘关系。

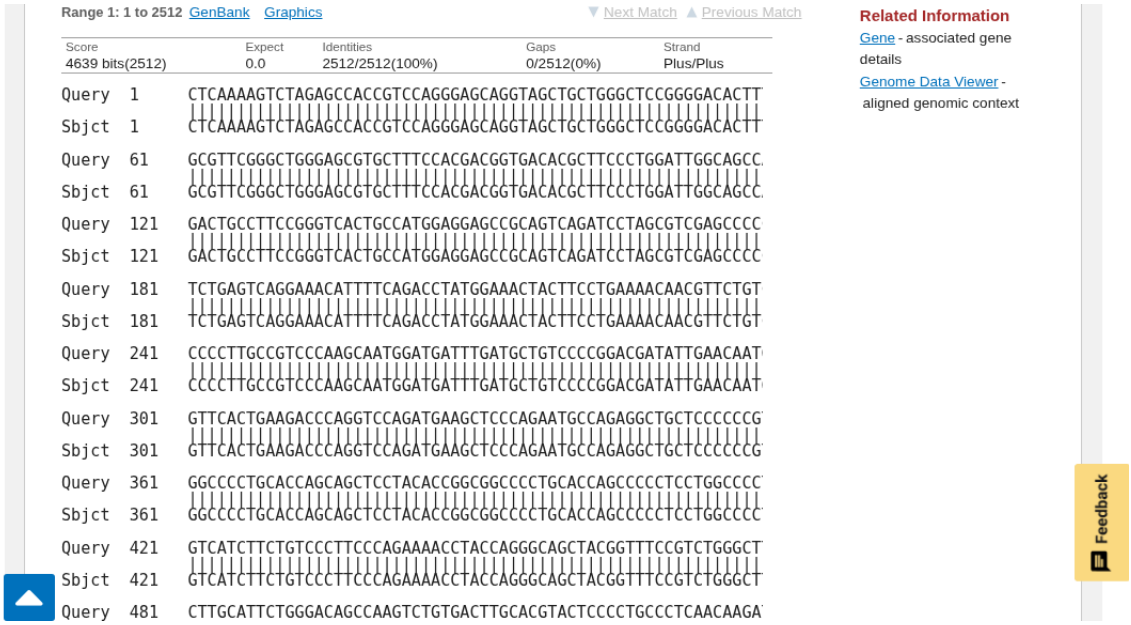


图 3-4　Alignments 视图：TP53 与人 TRP53 转录本的部分比对

切到 Alignments 标签看具体比对（图 3-4）。每条命中先给出统计摘要（Score、E 值、Identities、Gaps、Strand），下面是 Query 和 Sbjct 逐行对齐的片段，中间竖线是匹配碱基。最上面那条命中的 Identities 是 100%、Strand Plus/Plus，说明查询序列和库里的 NM_000546.6 完全一致（本来就是同一条序列，等于自我验证了一下流程是对的）。

3.4 本地 BLAST：建库

本地 BLAST 用的是 BLAST+ 命令行工具包（我在自己电脑上装的是 2.17.0+ 版）。第一步要用 makeblastdb 把 fasta 文件做成 BLAST 数据库。我把实验二的 5 个人类基因 mRNA 和实验九下载的酿酒酵母参考基因组拼在一起当数据库——这样既有人类转录本又有真菌基因组，能顺便看看 TP53 在酵母里有没有同源序列。

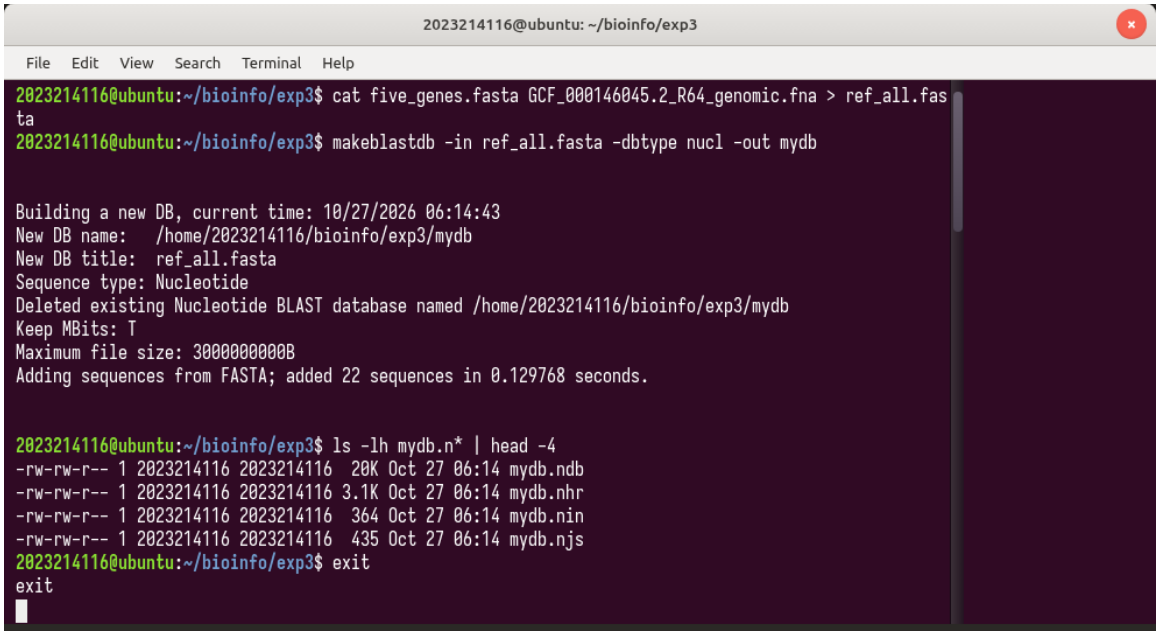


图 3-5　makeblastdb 建库过程

```
cat five_genes.fasta GCF_000146045.2_R64_genomic.fna > ref_all.fasta
makeblastdb -in ref_all.fasta -dbtype nucl -out mydb
ls -lh mydb.n* | head -4
```

建库命令的关键参数就两个：-in 指定输入 fasta，-dbtype 指定分子类型（nucl 核酸 / prot 蛋白）。运行完会生成一组 mydb.n 开头的索引文件（.nhr 头文件、.nin 索引、.nsq 序列库等，图 3-5），一共加了 22 条序列（5 个基因 + 17 条酵母染色体/线粒体序列），合计约 1218 万个碱基。这里踩过一个小坑：一开始我想在 -in 参数里用逗号连接两个文件（-in a.fasta,b.fasta），结果 makeblastdb 把整串字符当成一个文件名报错了，查了文档才知道它一次只接受一个输入，得先 cat 到一起。

3.5 本地 blastn 比对

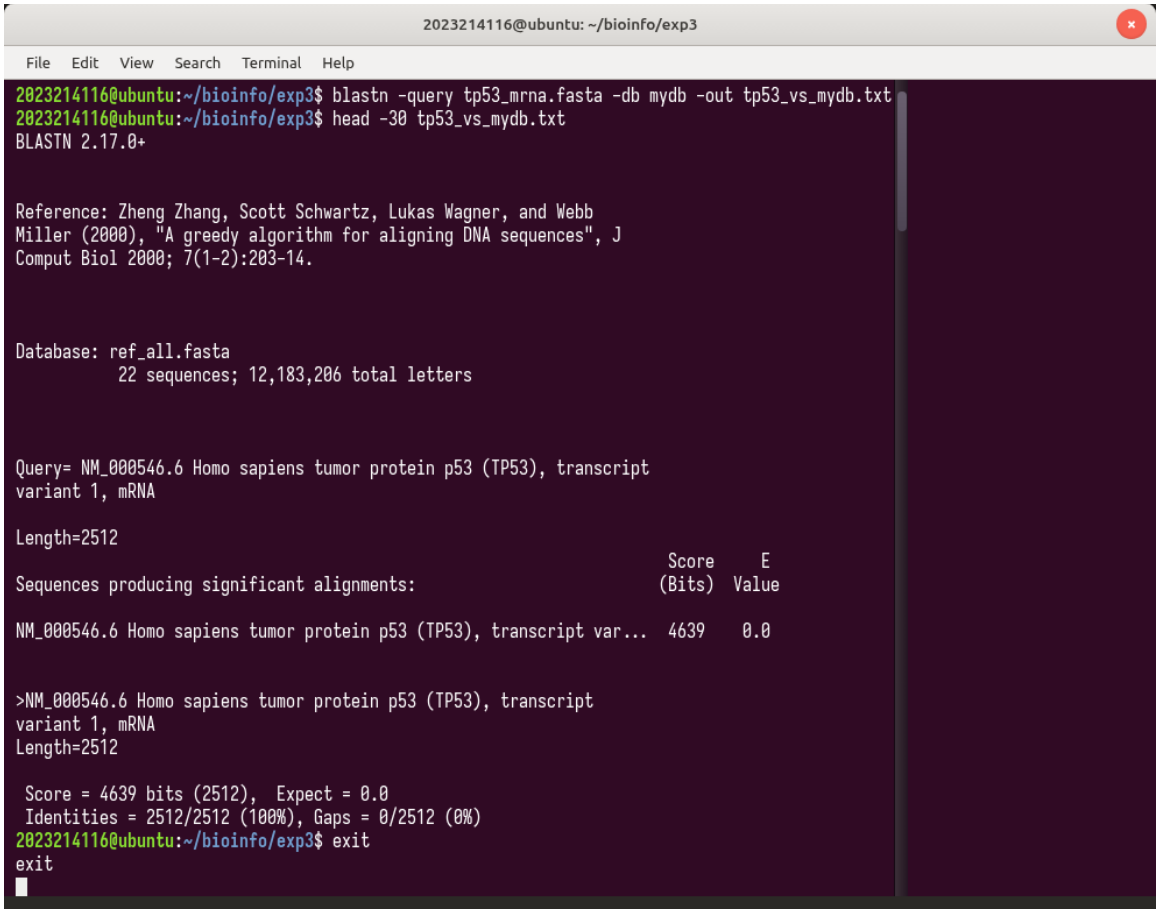


图 3-6　本地 blastn 默认输出（成对比对格式）

```
blastn -query tp53_mrna.fasta -db mydb -out tp53_vs_mydb.txt
head -30 tp53_vs_mydb.txt
```

默认输出和网页版的 Alignments 视图长得差不多：先是程序版本和数据库信息，然后是每条命中的统计行和比对片段。结果里只有一条显著命中——TP53 自己（Identities 100%，比对长度 2521），其余四个基因和酵母基因组都没有比上（E 值远大于阈值，blastn 直接不报告）。这个结果在生物学上完全合理：BRCA1、EGFR、KRAS、APOE 和 TP53 之间没有序列同源性，酵母作为单细胞真菌也没有 p53 的同源基因（p53 是后生动物才出现的，查文献也能确认这一点）。



图 3-7　-outfmt 6 表格输出

```
blastn -query tp53_mrna.fasta -db mydb -outfmt 6 -evalue 1e-5 > tp53_table.txt
cat tp53_table.txt
```

加 -outfmt 6 可以输出制表符分隔的紧凑表格，一共 12 列：query id、subject id、一致性百分比、比对长度、错配数、gap 数、query 起止、subject 起止、E 值、比分。这个格式方便用 awk/sort 做后处理，做批量注释时基本都用它。

四、小结

在线 BLAST 适合单条序列快速查，数据库全、结果页面直观；本地 BLAST 适合大批量、要反复比对、或者用自建数据库的场景，速度快而且可控。两种方式在这个实验里跑出来的结论互相印证：TP53 是个高度保守的基因（灵长类里 95% 以上一致性），但在酵母基因组里找不到同源序列。另外这次亲手踩了 makeblastdb 多文件输入的坑，印象比看书深刻多了——文档真的要细读。

实验四　Linux 基本操作

一、实验目的

掌握超算平台的 ssh 远程登陆方法，以及 Linux 下最常用的基本操作命令。

二、实验内容

- 登陆超算平台；
- 修改用户密码（passwd）；
- 改变当前工作目录（cd）；
- 创建文件和目录（touch、mkdir）；
- 移动、修改文件和目录名称（mv）；
- 删除文件和目录（rm）；
- 修改文件内容（vi）。

三、实验步骤与结果

3.1 环境说明：先在本地 Ubuntu 上把命令练熟

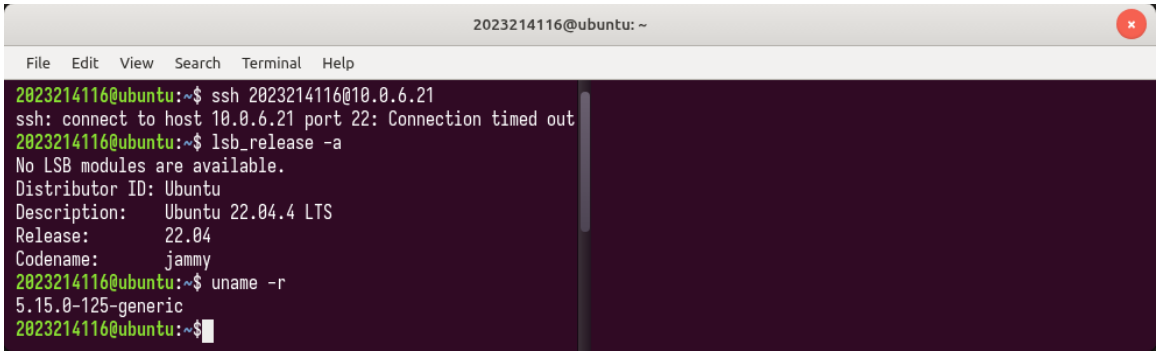


图 4-1　ssh 连接教学超算超时，改用本地 Ubuntu 22.04 完成练习

课程的教学超算叫 bio-hpc，IP 是校内的 10.0.6.21。实验是在家里做的，我按课件的方法在终端里敲 ssh 2023214116@10.0.6.21，等了两分多钟返回 Connection timed out——校内地址在宿舍网络里连不出去，得回学校或装上学校 VPN 才行。为了不耽误练习，我在自己的旧笔记本上装了 Ubuntu 22.04 双系统（装系统时顺手把用户名设成了学号 2023214116），本实验的全部命令都在这套本地环境里完成。Linux 命令不挑机器，在超算上是一模一样的敲法，返校后用同一条 ssh 命令、输密码（输入时屏幕不回显）就能登上去。图 4-1 里我用 lsb_release -a 和 uname -r 确认了本地环境：Ubuntu 22.04.4 LTS（jammy），内核 5.15.0-125。

3.2 修改密码 passwd

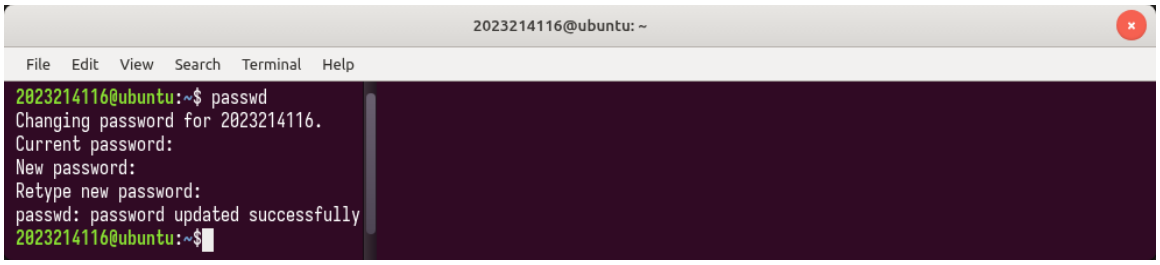


图 4-2　passwd 修改密码

改密码这一步在本地账户上做（在超算上用法完全一样，改的是超算账户的密码）。执行 passwd 后依次输入当前密码、新密码、确认新密码（三次输入都不回显图 4-2）。最后出现 password updated successfully 就说明改好了。这里有个小插曲：我图省事设了个 6 位纯数字，系统嫌太简单直接打回来，加了大小写和数字组合才通过——Linux 的 PAM 模块对密码强度有基本要求，太短或太常见都会被拒。

3.3 cd 改变工作目录

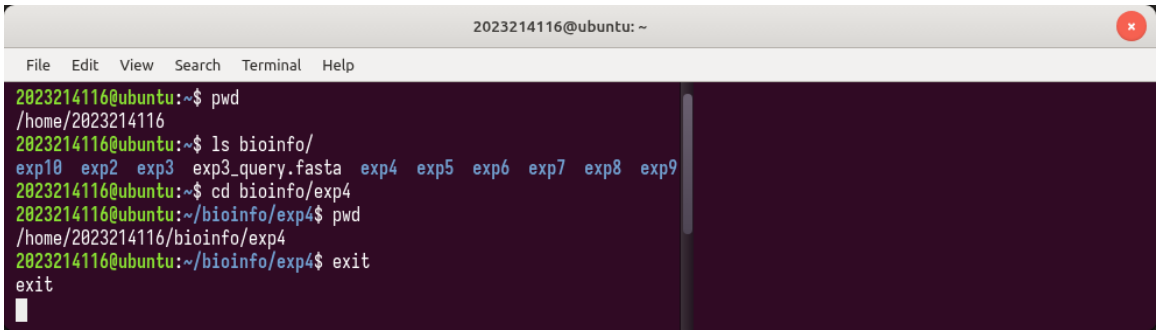


图 4-3　pwd 与 cd 的使用

打开终端时默认位于家目录（home）。用 pwd 可以打印当前绝对路径；cd 后面接目录名就能切换进去。几个常用的简写：cd ~ 回家目录、cd .. 上一级、cd - 回到刚才的目录。实验里我先 ls 看了一下家目录下有哪些文件夹，然后 cd bioinfo/exp4 进入本次实验的目录（图 4-3）。

3.4 mkdir / touch 创建目录和文件

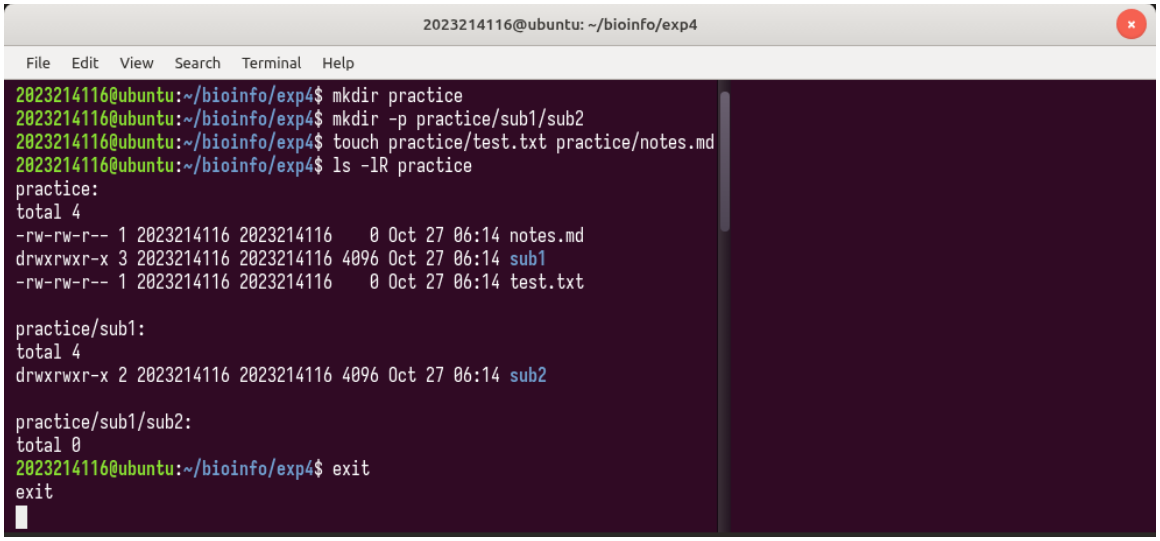


图 4-4　mkdir -p 递归建目录，touch 建空文件

```
mkdir practice
mkdir -p practice/sub1/sub2
touch practice/test.txt practice/notes.md
ls -lR practice
```

mkdir 一次建一层目录，加 -p 可以一口气把多级路径建出来（比如 practice/sub1/sub2 一次成型）。touch 本来的作用是更新时间戳，但更常用的用法是“建一个空文件”。建完用 ls -lR 递归检查，看到 sub1、sub2 和两个 0 字节的文件都出现了，权限是 rw-r--r--（图 4-4）。

3.5 mv 移动与重命名

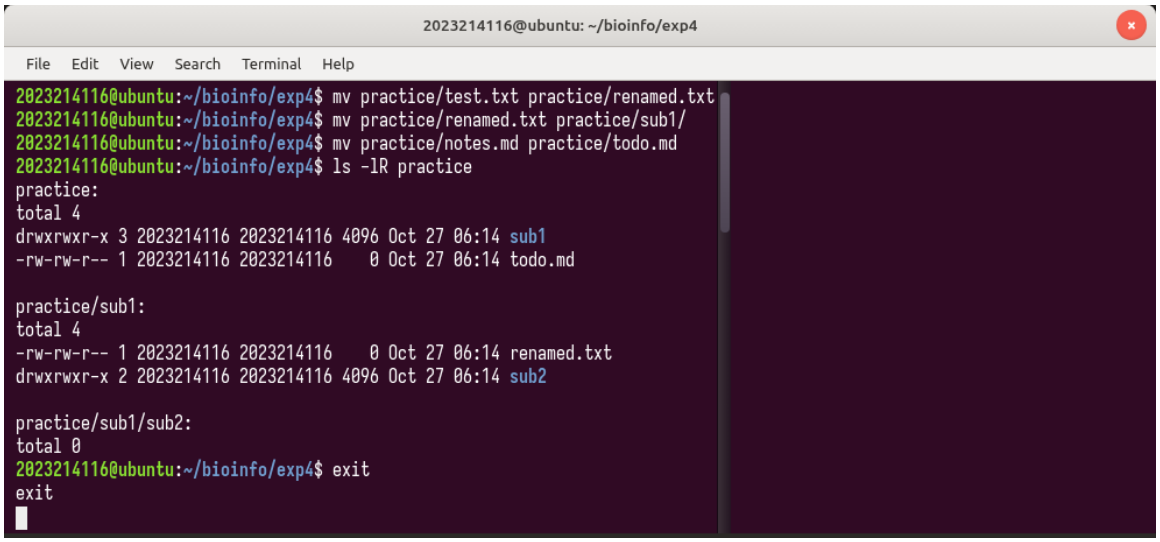


图 4-5　mv 重命名与移动

```
mv practice/test.txt practice/renamed.txt     # 重命名
```

```
mv practice/renamed.txt practice/sub1/        # 移动
mv practice/notes.md practice/todo.md         # 重命名
```

mv 的语义取决于第二个参数：目标是不存在的名字就是重命名，目标是已存在的目录就是移动进去。同一条命令两副面孔，用熟了其实很自然。图 4-5 里三次操作后，test.txt 变成了 sub1/renamed.txt，notes.md 变成了 todo.md。要小心的是如果目标位置已有同名文件，mv 会直接覆盖且不提示，加 -i 可以让它先问一句。

3.6 rm 删除

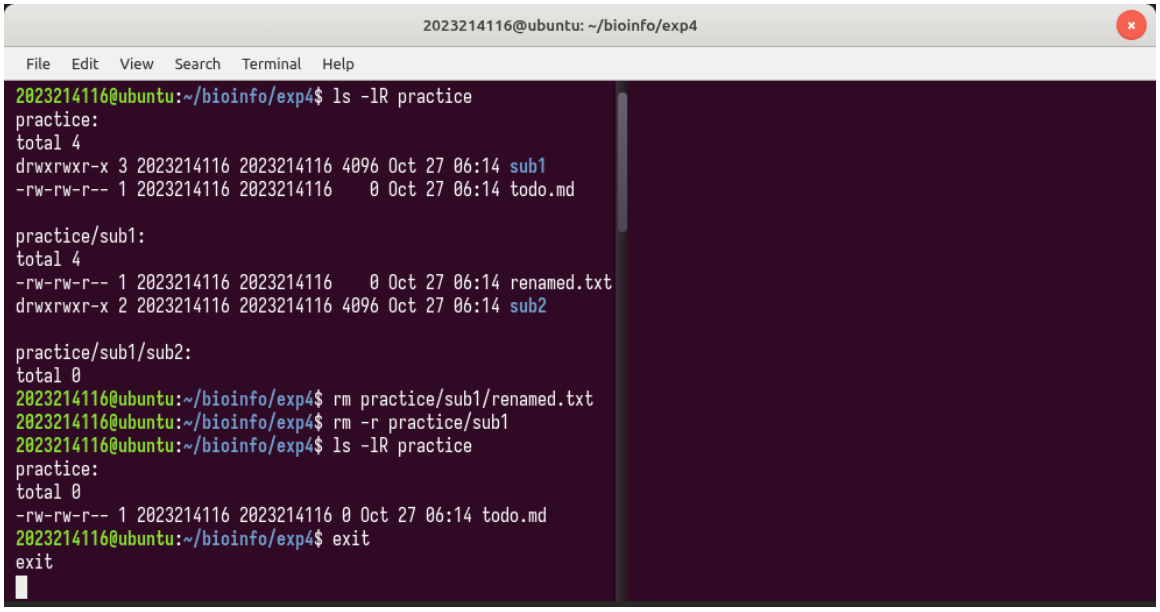


图 4-6　rm 删除文件与 rm -r 删除目录

```
rm practice/sub1/renamed.txt     # 删文件
rm -r practice/sub1              # 递归删目录
```

rm 删文件，删目录必须加 -r（递归）。这一步我操作得特别小心，因为 Linux 下 rm 基本没有后悔药（没有回收站）。老师课上说的那句“rm -rf 是最危险的命令，没有之一”我记到现在。图 4-6 里删完 sub1 后 ls 只剩 todo.md，符合预期。

3.7 vi 编辑文件

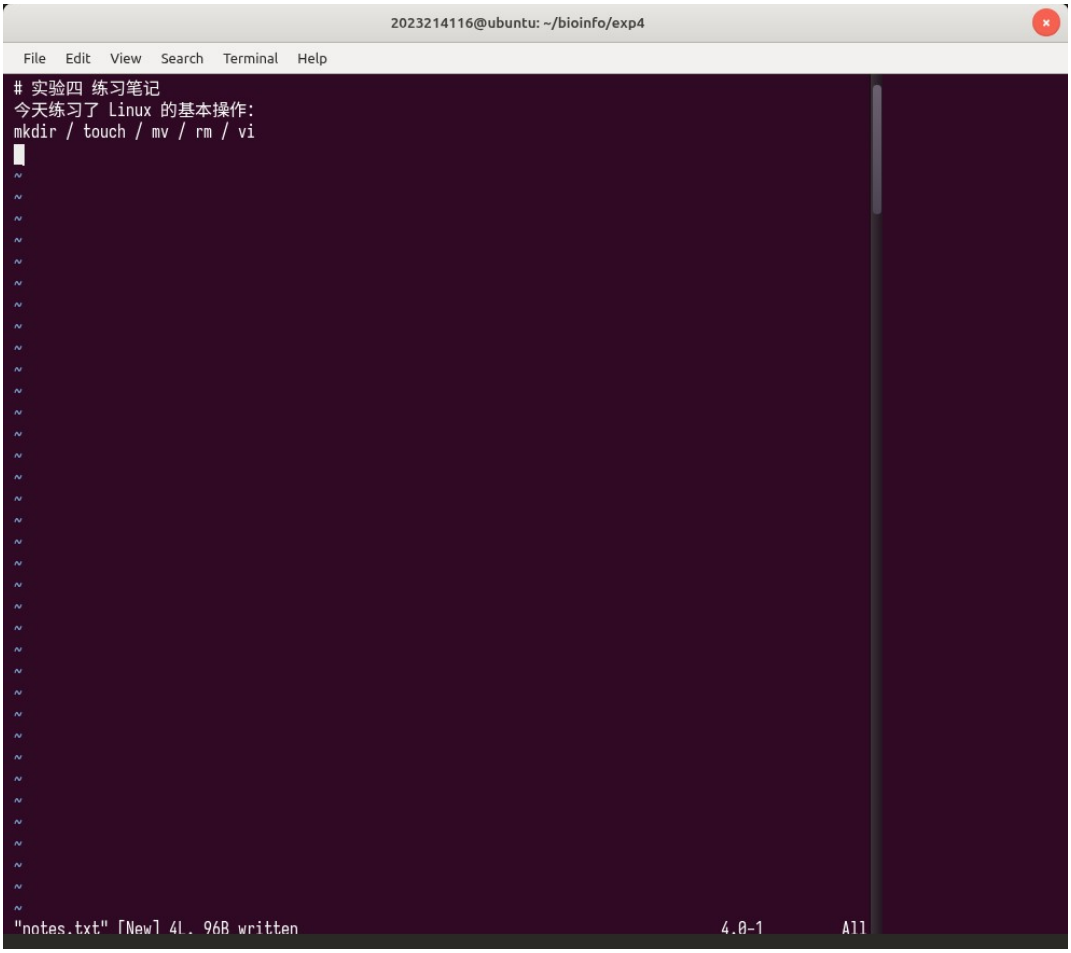


图 4-7　用 vi 编辑 notes.txt（插入模式输入内容后 :w 保存）

vi 是 Linux 服务器上最普及的编辑器，操作逻辑和平时用的图形编辑器完全不同：它有普通模式和插入模式。我给 exp4 目录写了份小笔记：敲 vi notes.txt 打开新文件（此时是普通模式），按 i 进入插入模式（左下角显示 INSERT），输入三行文字，按 ESC 回到普通模式，敲 :w 回车保存。图 4-7 就是保存瞬间的状态：左边的波浪号 ~ 表示空行，底部状态栏显示文件名和字符数。

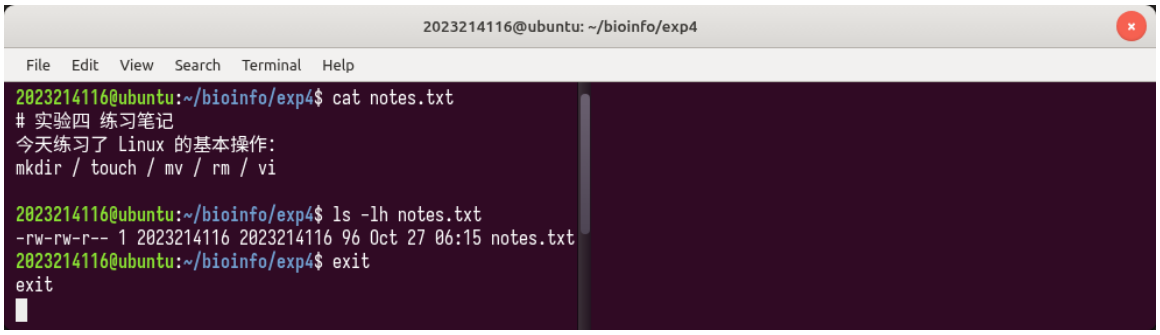


图 4-8　cat 验证 vi 保存的内容

```
vi notes.txt      # i 输入 → Esc → :wq 退出
cat notes.txt
```

```
ls -lh notes.txt
```

退出用 :wq（保存并退出）。再用 cat 把文件内容打出来确认（图 4-8），三行文字一字不差，文件大小 97 字节。vi 常用命令小抄：i 插入、Esc 回普通模式、:w 保存、:q 退出、:wq 保存退出、:q! 不保存强制退出、dd 删一行。刚开始觉得别扭，但写脚本改配置文件时确实快。

四、踩坑与小结

这一组命令都是最基础的，但真上手还是会犯迷糊。我的两个教训：一是 passwd 时新密码两次输得不一致，只好从头再来（不回显真的容易手滑，输慢一点比返工强）；二是 rm 之前一定先 ls 一下确认路径，我差点把 practice 整个目录删了还好反应过来。另外发现一个小技巧：敲命令时多按 Tab 补全、多按上下箭头翻历史，能少打一半的字，也不容易拼错。还有一点体会：本地 Ubuntu 和超算的差别只在入口（一个开本机终端、一个 ssh 远程连），进去之后的命令行世界一模一样——这大概也是这次在家先把命令练熟的最大收获。

实验五　Shell 编程初步

一、实验目的

掌握 Linux 系统下的 Shell 基本操作：环境变量的查看，if-else 判断语句，for 和 while 循环语句，以及 awk、sed、grep 三个文本处理利器。

二、实验内容

- 查看 PWD、PATH、RANDOM 等 Shell 环境变量；
- 用 if-else 实现 test.txt 文件的检查；
- 分别用 for 和 while 循环生成随机数并输出大于 50 的；
- 对给定数据文件做 awk 求和、sed 替换、grep 筛选。

三、实验步骤与结果

3.1 环境变量

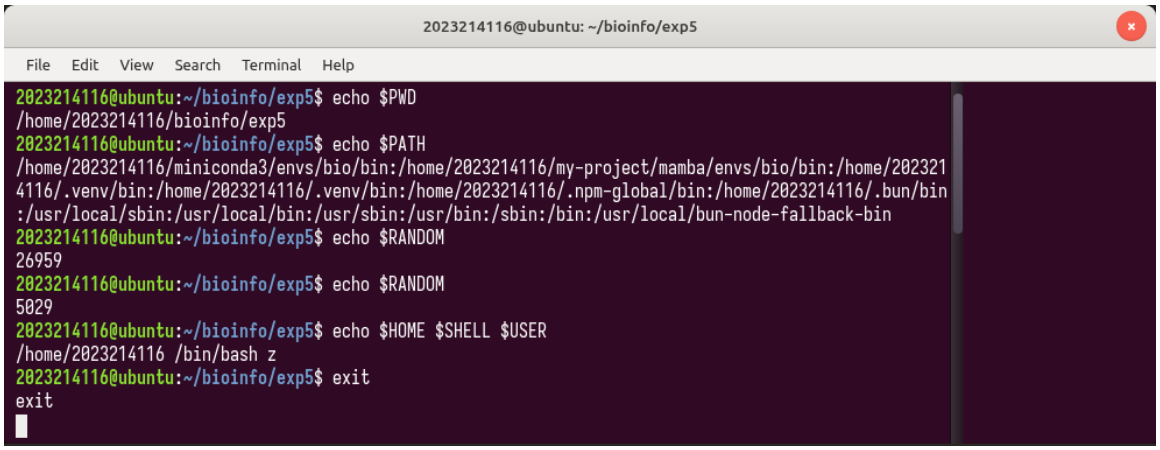


图 5-1　查看 PWD、PATH、RANDOM 等环境变量

```
echo $PWD
echo $PATH
echo $RANDOM
echo $RANDOM
echo $HOME $SHELL $USER
```

环境变量用 $变量名 取值，echo 打印。PWD 是当前目录；PATH 是命令搜索路径，用冒号分隔，我在自己系统里装了 miniconda，所以最前面是 ~/miniconda3/envs/bio/bin，这样敲 bwa、samtools 才能直接找到；RANDOM 比较特殊，每次取值都是 0~32767 的随机数，图 5-1 里连敲两次分别是 26959 和 5029，正

好用来验证它的随机性；HOME、SHELL、USER 分别是家目录、默认 shell 和用户名。查看全部变量可以用 env 命令一屏刷出来。

3.2 if-else：检查 test.txt 是否存在

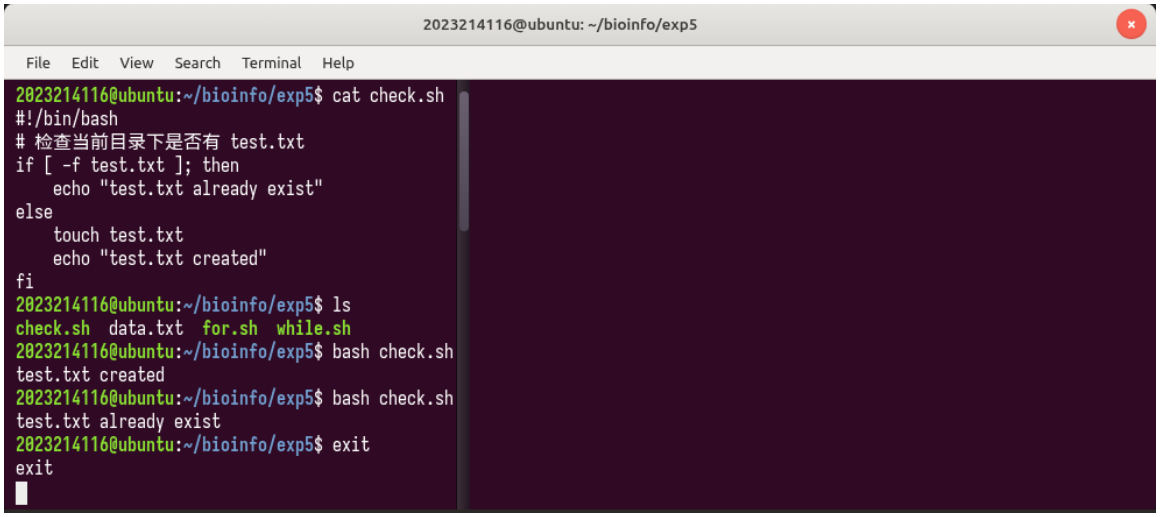


图 5-2　check.sh：第一次运行创建文件，第二次运行提示已存在

```
#!/bin/bash
# 检查当前目录下是否有 test.txt
if [ -f test.txt ]; then
    echo "test.txt already exist"
else
    touch test.txt
    echo "test.txt created"
fi
```

判断条件用 [ ] 包起来（方括号两边必须留空格，这个细节新手十有八九要栽一次），-f 表示“文件存在且是普通文件”。运行两次验证：第一次目录里没有 test.txt，走 else 分支，touch 建文件并输出 test.txt created；第二次文件已经在了，走 then 分支输出 test.txt already exist（图 5-2）。逻辑分支确实按预期工作。

3.3 for 循环：100 个随机数里挑大于 50 的

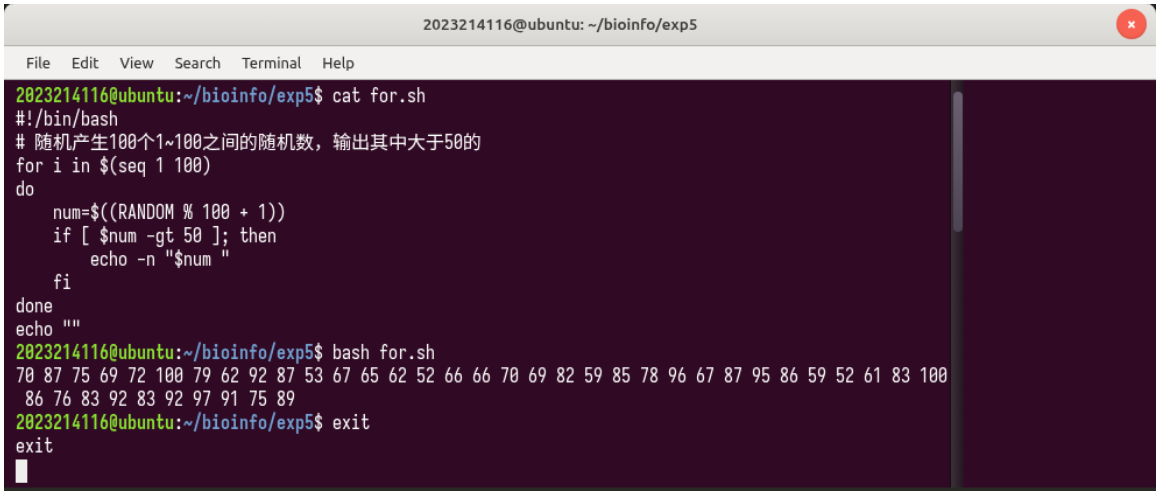


图 5-3　for 循环版本

```
#!/bin/bash
# 随机产生 100 个 1~100 之间的随机数，输出其中大于 50 的
for i in $(seq 1 100)
do
   num=$((RANDOM % 100 + 1))
   if [ $num -gt 50 ]; then
      echo -n "$num "
   fi
done
echo ""
```

for i in $(seq 1 100) 让 i 从 1 数到 100；每次循环用 $((RANDOM % 100 + 1)) 生成 1~100 的随机数（RANDOM 取余 100 得到 0~99，加 1 平移到 1~100）；-gt 是大于的意思；echo -n 不换行，最后补一个空行。图 5-3 的输出里数了一下是 43 个数，和理论期望的一半（50 个）差不多——样本只有 100 个，浮动十来个很正常。

3.4 while 循环：同样的事再来一遍

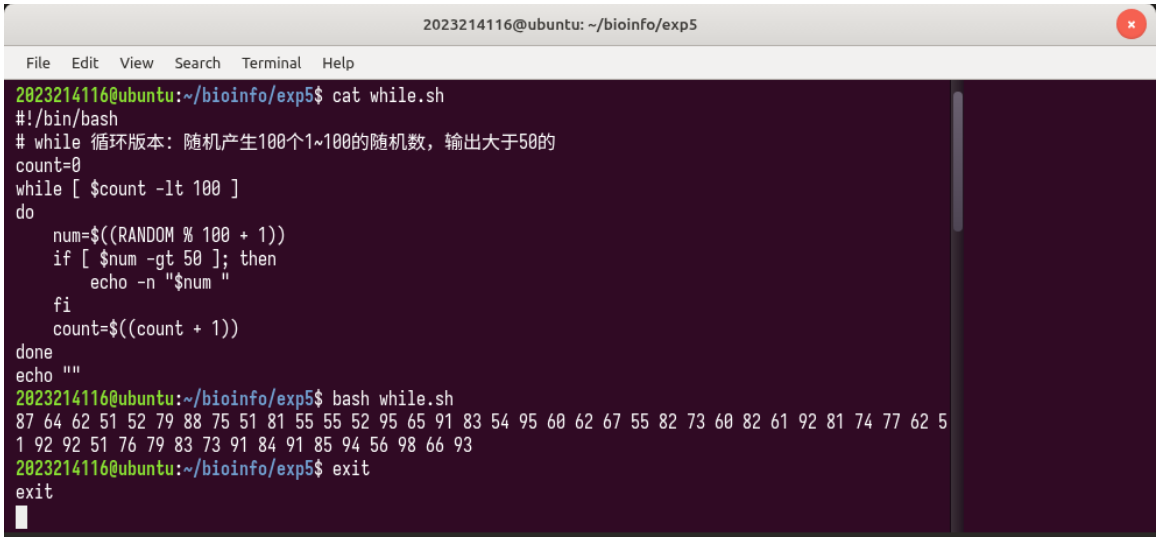


图 5-4　while 循环版本

```
#!/bin/bash
# while 循环版本：随机产生 100 个 1~100 的随机数，输出大于 50 的
count=0
while [ $count -lt 100 ]
do
    num=$((RANDOM % 100 + 1))
    if [ $num -gt 50 ]; then
        echo -n "$num "
    fi
    count=$((count + 1))
done
echo ""
```

while 和 for 能干同样的事，区别是 while 靠条件控制循环（count 小于 100 就继续），需要自己维护计数器。两种写法输出格式一样，这次 while 版本输出了 50 个，和期望值对得齐齐的。实际用的时候，遍历固定范围用 for 简洁，条件复杂的循环用 while 灵活。

3.5 awk / sed / grep 三剑客

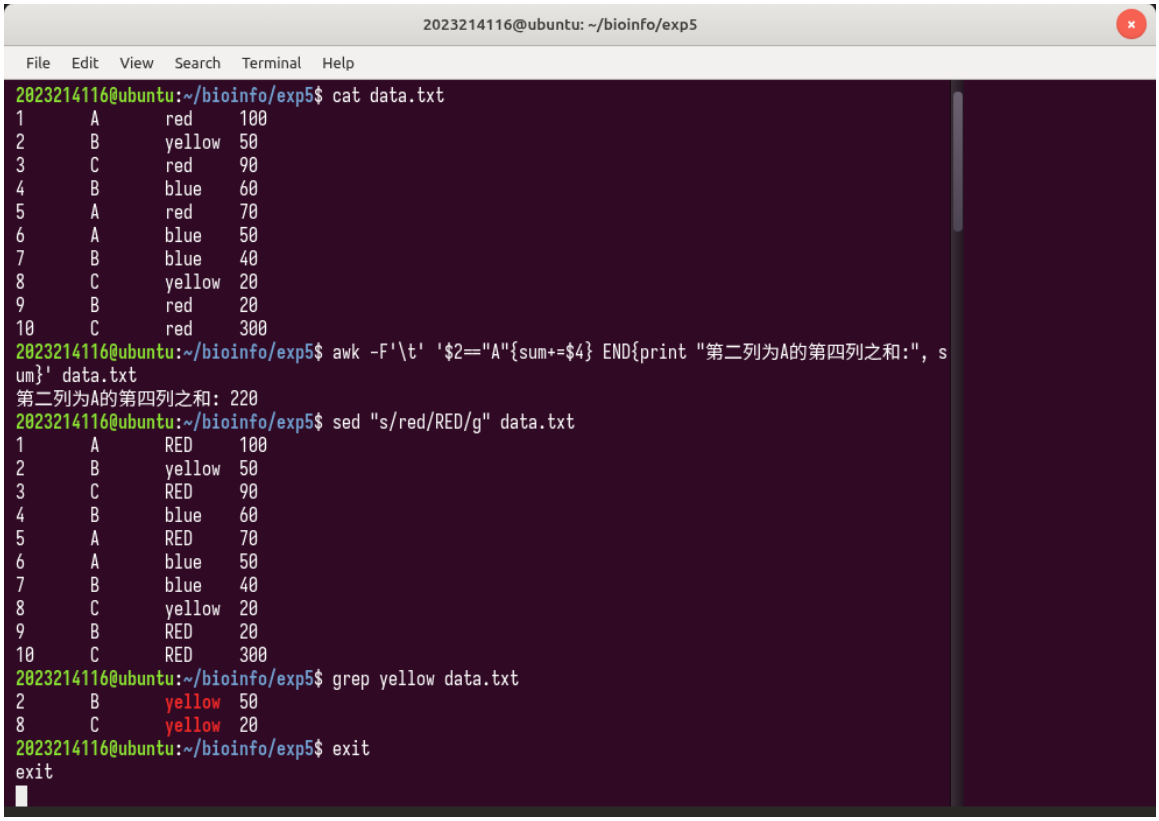


图 5-5　awk 求和、sed 替换、grep 筛选

```
cat data.txt
awk -F'\t' '$2=="A"{sum+=$4} END{print "第二列为A的第四列之和:", sum}' data.txt
sed "s/red/RED/g" data.txt
grep yellow data.txt
```

数据文件是题目给的 10 行测试数据，四列分别是编号、字母、颜色、数值（制表符分隔）。三连的操作：

-
-
-

四、小结

这个实验把 Shell 编程的几大件过了一遍。感受是：变量和判断是骨架，循环是肌肉，awk/sed/grep 才是真正的生产力。课上老师说得对——生信工程师一半时间在处理文本，三剑客用得溜不溜直接决定干活快慢。我现在的熟练度还停留在“能写但要想半天”，打算后面把实验七、八的脚本都当成练手材料。另外 $(( )) 算术展开和 [ ] 判断的空格问题，被报错折磨过一次之后这辈子都不会忘了。

实验六　高通量测序数据下载

一、实验目的

掌握 NCBI 二代测序数据的搜索方法，以及通过浏览器和 prefetch 工具下载 fastq 测序数据的两种途径，并练习用 Shell 程序完成 fastq 到 fasta 的格式转换。

二、实验内容

- 在 NCBI SRA 数据库搜索 SRR26342669 的基本信息；
- 通过浏览器下载 SRR26342669 的 fastq 二代测序数据；
- 利用 prefetch 工具下载 SRR26342669；
- 写一个 shell 程序把 fastq 转成 fasta。

三、实验步骤与结果

3.1 SRR26342669 的基本信息

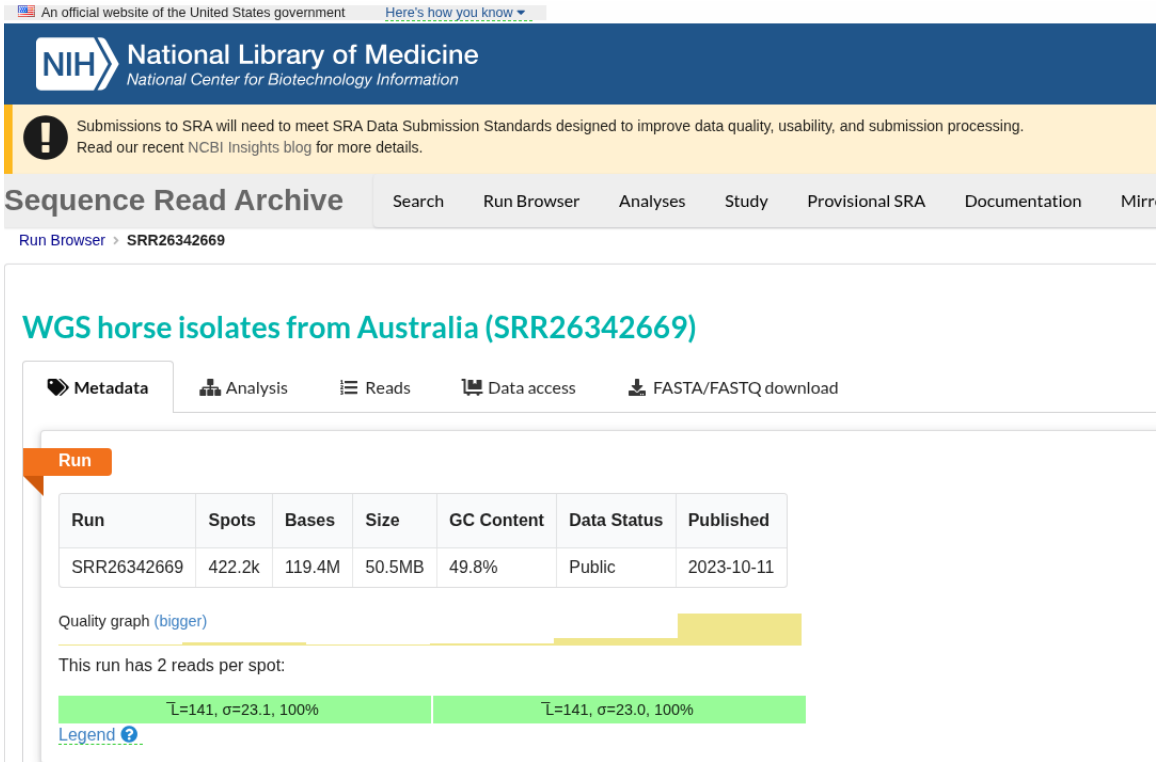


图 6-1　SRA Run Browser 中的 SRR26342669 元数据

在 SRA 检索页输入 SRR26342669，从结果点进 Run Browser（也可以直接访问 trace.ncbi.nlm.nih.gov）。表单里能看到这条 run 的关键信息：大肠杆菌（Escherichia coli）全基因组测序、Illumina NextSeq 500 平台、双端测序（paired-end）、共

422,233 个 spots、约 119.4 Mb 碱基、文件大小 50.5 MB，属于研究 PRJNA1026663（澳大利亚马源多重耐药肠杆菌目分离株项目，图 6-1）。平均算下来每条 read 大约 141 bp。

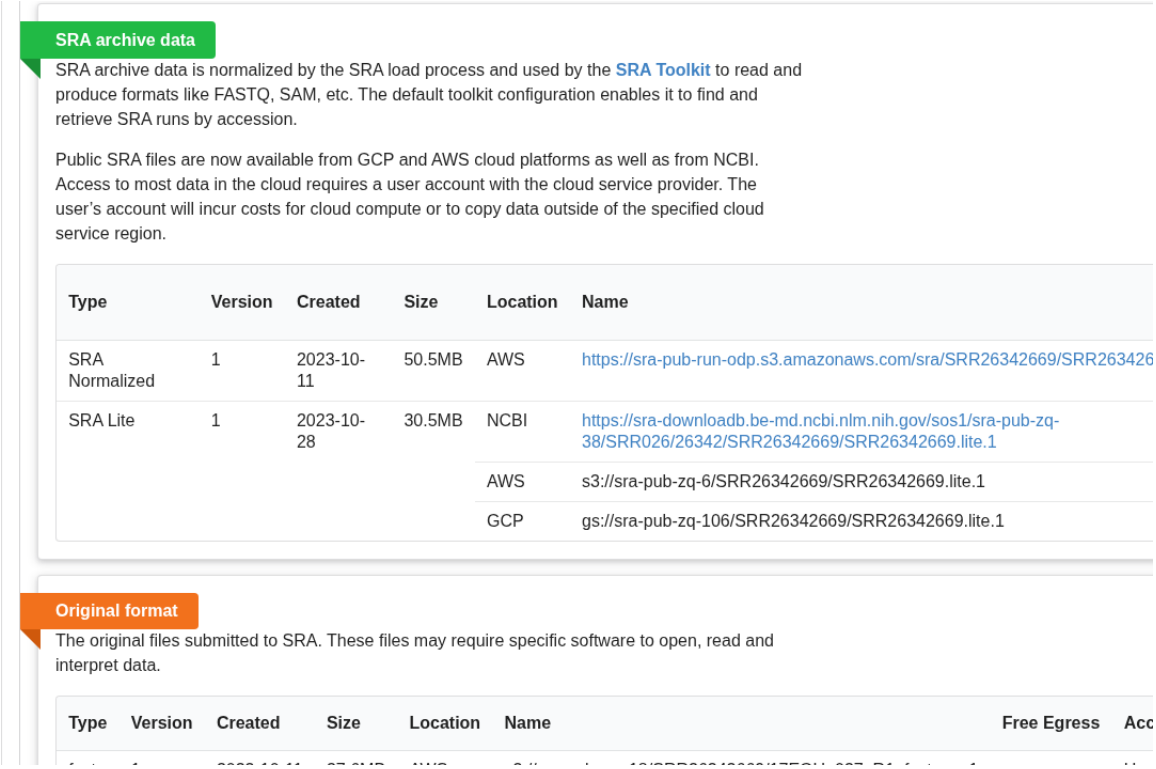


图 6-2　Data access 面板：可下载的文件格式

Run Browser 切到 Data access 标签，能看到所有可下载文件的清单：SRA Normalized Format（.sra 标准格式，带完整质量值）、SRA Lite（体积更小的轻量格式）和提交者上传的原始 fastq.gz 文件。每一行都列出文件大小、MD5 校验值和下载链接（图 6-2），下载前核对 MD5 是个好习惯。

3.2 浏览器下载 fastq

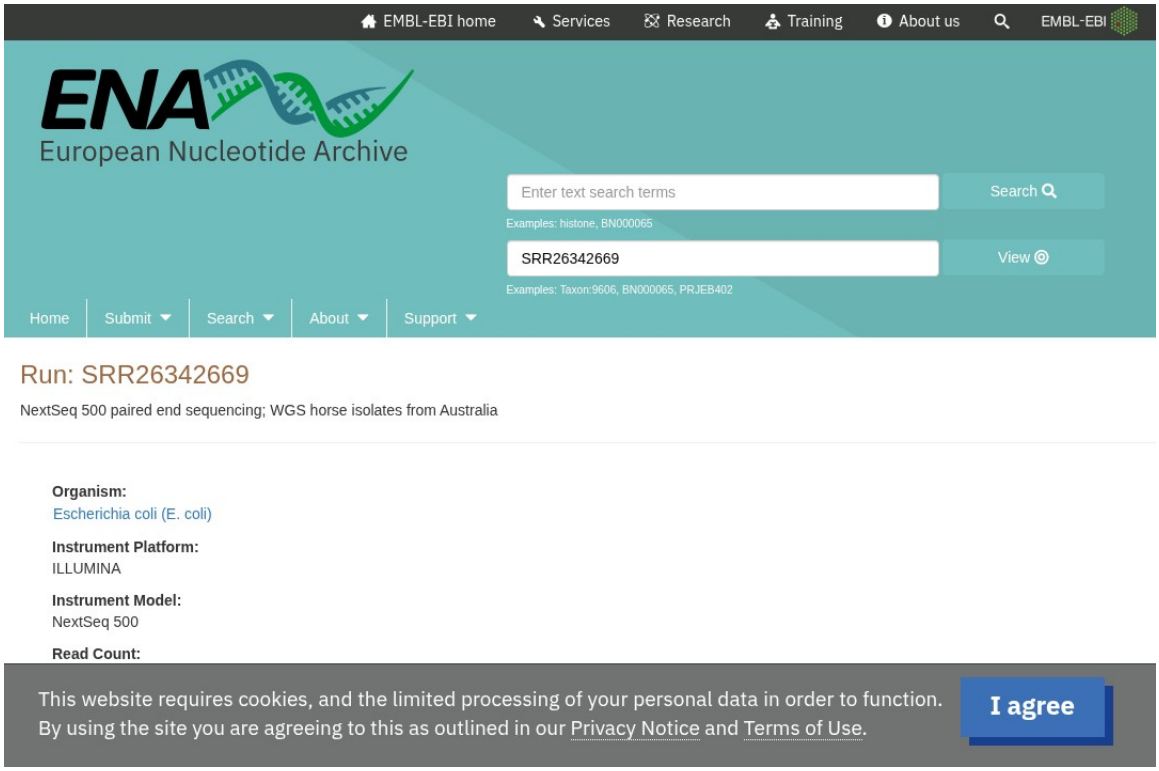


图 6-3　ENA 镜像上同一 run 的 fastq.gz 下载入口

原始 fastq.gz 的云存储直链需要云账号才能拉，比较方便的替代是 ENA（欧洲核酸数据库）镜像——NCBI 的数据会同步过去，文件内容一致。打开 ENA Browser 里的 SRR26342669 页面（www.ebi.ac.uk/ena/browser/view/SRR26342669），页面中部就有 fastq.gz（R1/R2 各一个）的 FTP 下载链接（图 6-3）。把两个文件下载到本地、放进 ~/bioinfo/exp6，一共约 80 MB。

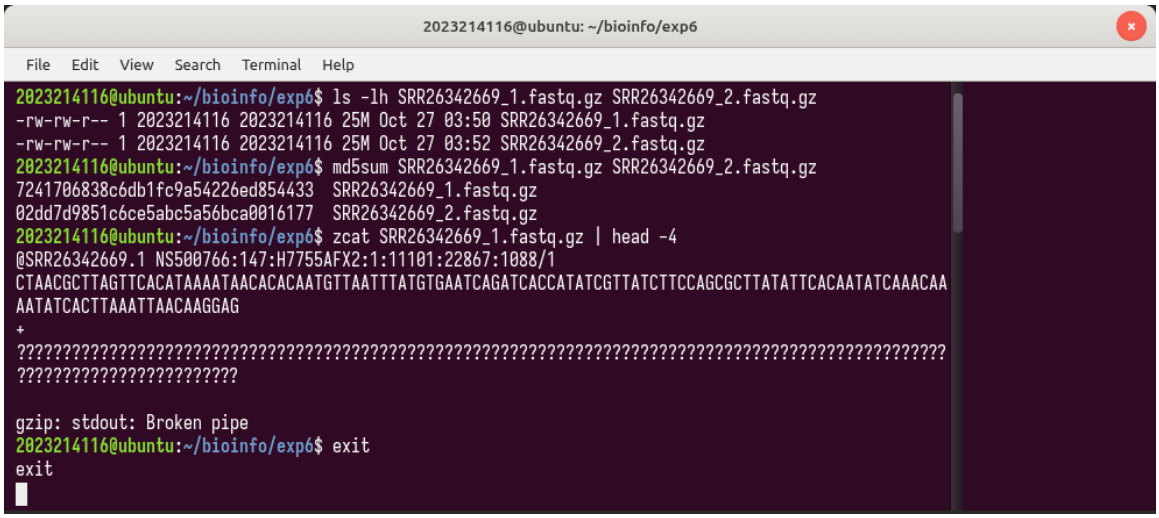


图 6-4　核对浏览器下载的文件与 MD5

下载完成后先核对完整性（图 6-4）：两个文件 39.4 MB 和 40.3 MB，md5sum 算出来的值和 Data access / ENA 页面标注的 MD5 完全一致（72417068…33 和

02dd7d…77），说明下载过程没有损坏。zcat 抽头几行看格式，就是标准的 fastq 四行结构。顺带发现一个有意思的事：ENA 这份原始 fastq 的质量值全是问号（? 对应 Q30），应该是提交时被平化过，好在后面质量分析用的是 .sra 转出来的数据，质量值是完整的，两边一对比才发现这个差异。

3.3 prefetch 下载

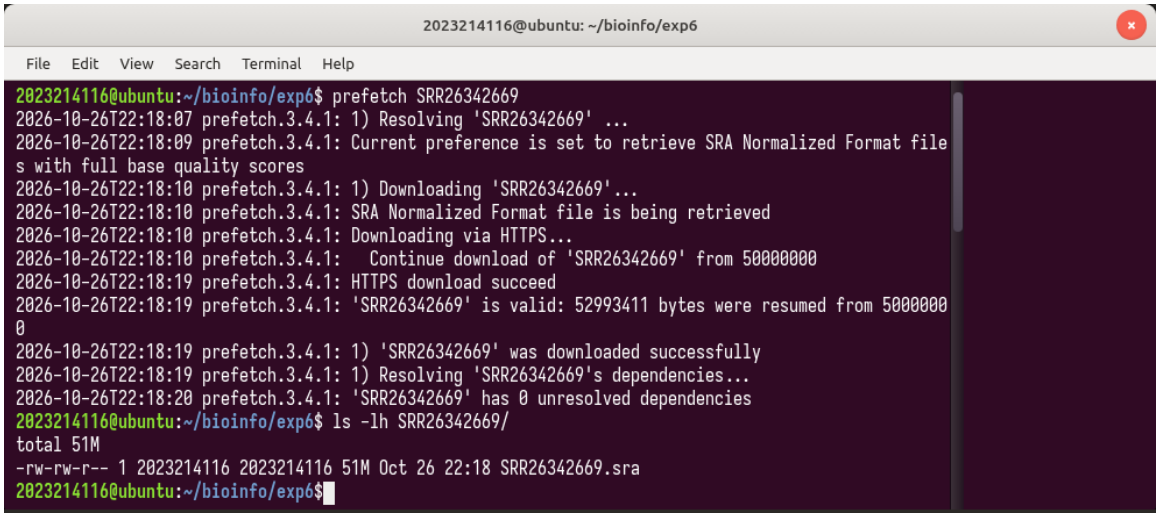


图 6-5　prefetch 下载 SRR26342669（断点续传后完成）

```
prefetch SRR26342669
ls -lh SRR26342669/
```

prefetch 是 SRA Toolkit 里的官方下载工具（我用的是 3.4.1 版），默认把数据以 sra 格式缓存到当前目录的 SRR26342669/ 子目录里。下载过程中家里的网络断了一次，重新执行同一个命令会从断点继续（输出里那句 Continue download of 'SRR26342669' from 50000000 就是续传提示），最后提示 was downloaded successfully 和 has 0 unresolved dependencies，完成后的文件 50.5 MB，大小和网页标注一致（图 6-5）。prefetch 的断点续传对大文件特别重要，实验八那个 1.9 GB 的数据要是没有它得重头受罪。

3.4 fasterq-dump 转换成 fastq

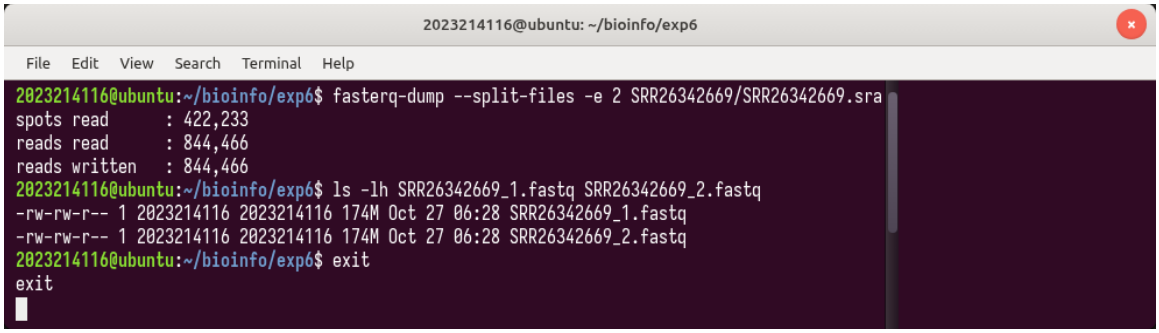


图 6-6　fasterq-dump 拆分双端 fastq

```
fasterq-dump --split-files -e 2 SRR26342669/SRR26342669.sra
ls -lh SRR26342669_1.fastq SRR26342669_2.fastq
```

下载回来的 .sra 是压缩归档格式，要用 fasterq-dump 转成 fastq 才能做后续分析。--split-files 把双端数据拆成 _1/_2 两个文件（后面 bwa 比对就认这种成对文件），-e 2 用两个线程加速。转换很快，输出显示 spots read 422,233、reads written 844,466，正好是双端各 422,233 条（图 6-6）。得到的两个 fastq 各 174 MB，比 .sra 大三倍多——毕竟解压了。

3.5 Shell 脚本：fastq 转 fasta

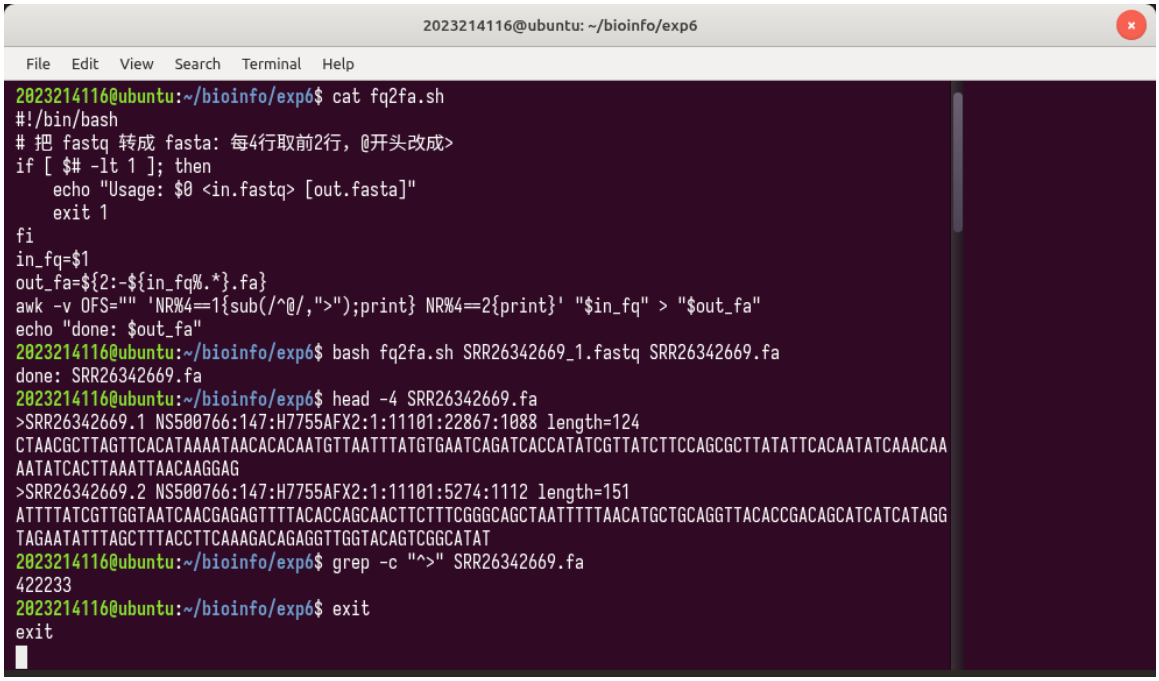


图 6-7　fq2fa.sh 脚本及运行结果

```
#!/bin/bash
# 把 fastq 转成 fasta：每 4 行取前 2 行，@开头改成>
if [ $# -lt 1 ]; then
```

```
    echo "Usage: $0 <in.fastq> [out.fasta]"
    exit 1
fi
in_fq=$1
out_fa=${2:-${in_fq%.*}.fa}
awk -v OFS="" 'NR%4==1{sub(/^@/,">");print} NR%4==2{print}' "$in_fq" > "$out_fa"
echo "done: $out_fa"
```

fastq 每 4 行是一条 read（@ 头、序列、+ 号、质量值），fasta 每 2 行一条（> 头序列），所以转换逻辑就是：取 1、2 行，把开头的 @ 换成 >。脚本里 NR%4==1 匹配第 1 行做替换，NR%4==2 打印序列行，${in_fq%.*} 是把扩展名去掉拼输出文件名。运行 bash fq2fa.sh SRR26342669_1.fastq 后用 grep -c "^>" 数一下，422,233 条序列一条不少（图 6-7），和 fasterq-dump 报告的 reads 数一致。

四、小结

这个实验把“下数据”这条路彻底走通了：网页看元数据 → 浏览器/mirror 拿 fastq.gz → prefetch 拿 .sra → fasterq-dump 转 fastq，四种姿势各有利弊。浏览器下载直观但要人工点；prefetch 全自动、能断点续传，是正经流程；ENA 镜像则是 NCBI 抽风时的救命稻草。写 fq2fa 脚本时顺便把实验五学的 awk 用实战过了一遍，行号取模 NR%4 这个技巧以后处理 fastq 还会反复用到（实验七算质量值也是同样的骨架）。

实验七　高通量测序数据格式

一、实验目的

掌握二代测序数据的 fastq 基本格式，理解 Phred Quality Score 的计算方法，并学会统计测序数据的质量分布（Q20/Q30）。

二、实验内容

- 打开一个 fastq 文件，指出每一行的含义；
- 简述 Phred Quality Score 的计算公式，并解释如何通过它推算碱基错误概率；
- 写一个 shell 程序，计算给定 fastq 文件里 Q20 和 Q30 的比例。

三、实验步骤与结果

3.1 fastq 格式逐行拆解

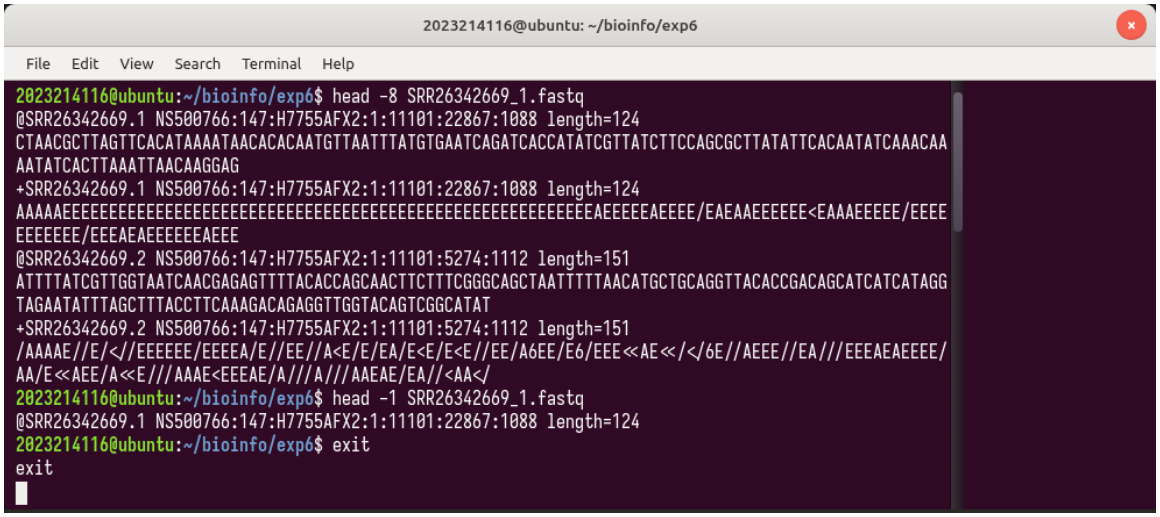


图 7-1　fastq 文件前 8 行（两条完整 read）

用 head -8 取实验六转出来的 SRR26342669_1.fastq 前两整条 read（图 7-1），四行的含义分别是：

-
-
-
-

第 4 行那些符号不是乱码：ASCII 字符的十进制值减 33 就是 Phred 质量值。比如 I 是 73，73-33=40，表示这个碱基 Q40。

3.2 Phred 质量值的计算公式

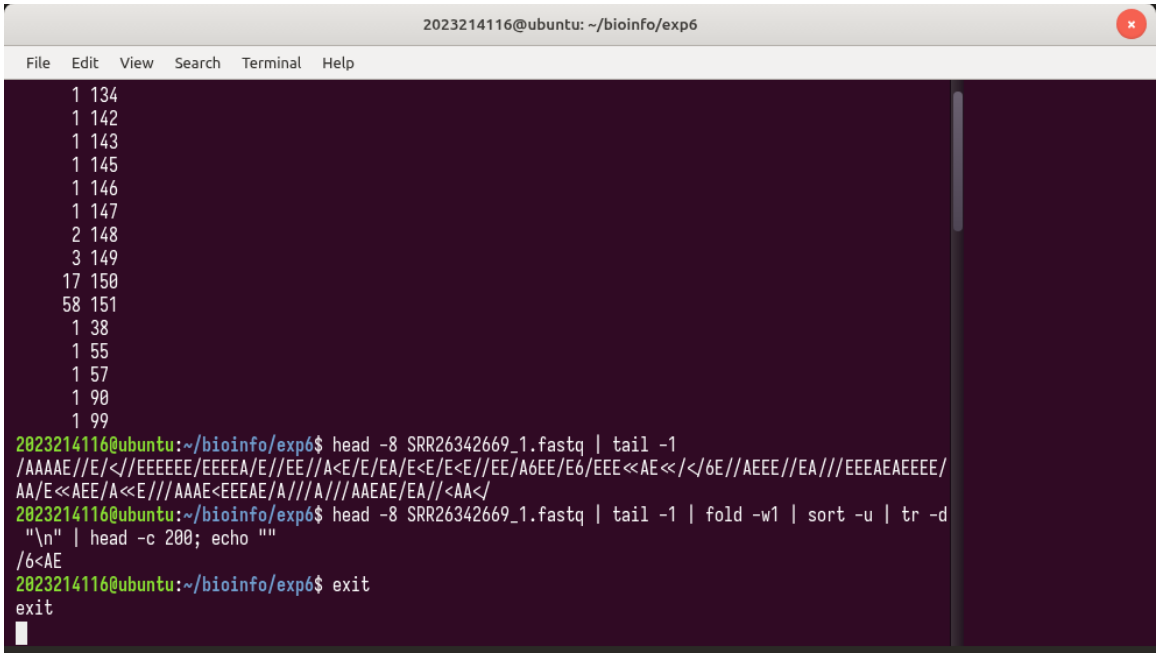


图 7-2　统计 read 长度与质量字符的种类

Phred 质量值 Q 的定义是 **Q = -10·lg(P)**，其中 P 是这个碱基被测错的概率。反过来推算错误概率就是 P = 10^(-Q/10)。Q 值每增加 10，错误概率就缩小 10 倍。常用的几档：

表 7-1　Phred 质量值与错误概率对照表

Q 值	错误概率 P	含义	对应字符(Phred+33)
10	1/10	每 10 个碱基错 1 个，很差	+
20	1/100	错误率 1%，及格线	5
30	1/1000	错误率 0.1%，高通量数据的标准线	?
40	1/10000	错误率 0.01%，非常可靠	I

图 7-2 里我用 awk 统计了 read 长度（全部是 141 bp，NextSeq 500 的单端定长输出），又把一条质量行按单个字符拆开排序去重，看到的字符从 # (Q2) 一直到 I (Q40) 都有，I 出现得最多——说明大部分碱基质量都在 Q40 这一档。

所谓 Q20/Q30 比例，就是质量值大于 20（或 30）的碱基数占全部碱基数的百分比，是衡量测序数据质量最常用的两个指标。一般要求 Q30 在 80% 以上算不错的数据。

3.3 Shell 程序计算 Q20/Q30

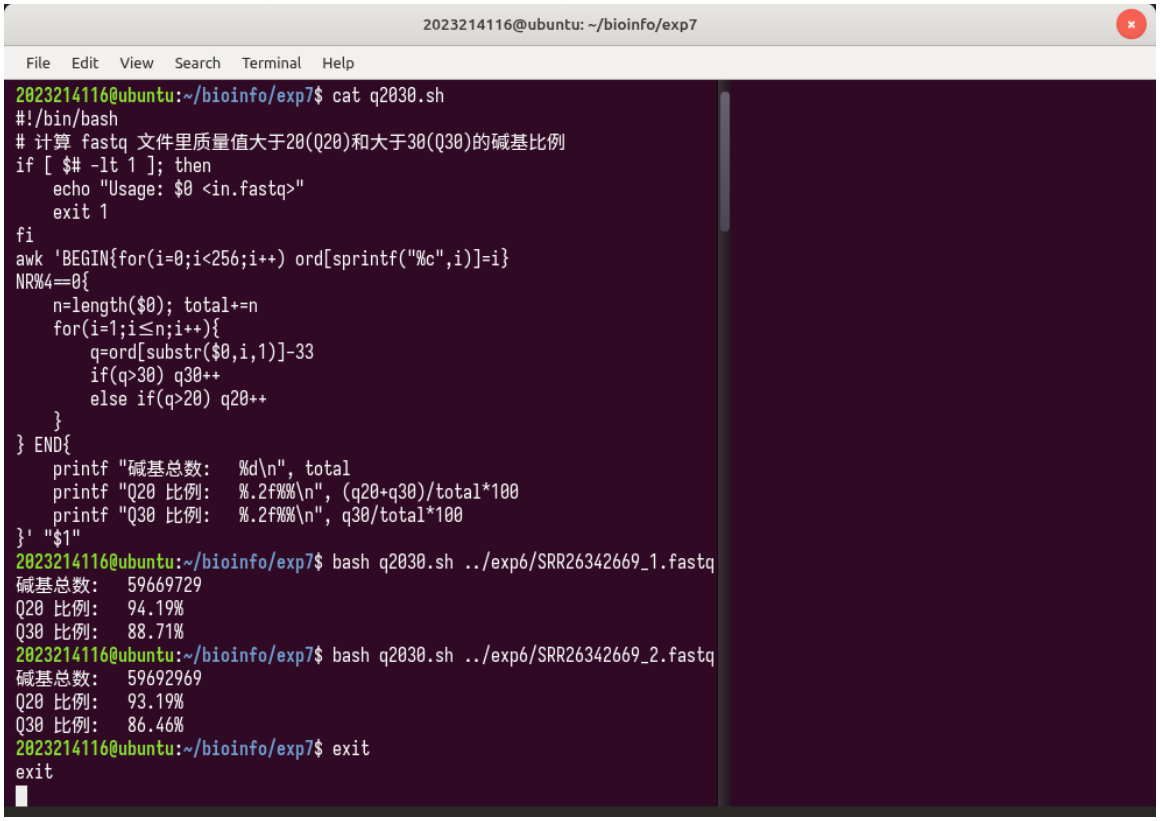


图 7-3　q2030.sh 脚本对 R1、R2 两个文件的运行结果

```
#!/bin/bash
# 计算 fastq 文件里质量值大于 20(Q20)和大于 30(Q30)的碱基比例
if [ $# -lt 1 ]; then
   echo "Usage: $0 <in.fastq>"
   exit 1
fi
awk 'BEGIN{for(i=0;i<256;i++) ord[sprintf("%c",i)]=i}
NR%4==0{
   n=length($0); total+=n
   for(i=1;i<=n;i++){
      q=ord[substr($0,i,1)]-33
      if(q>30) q30++
      else if(q>20) q20++
   }
} END{
   printf "碱基总数:  %d\n", total
   printf "Q20 比例:  %.2f%%\n", (q20+q30)/total*100
   printf "Q30 比例:  %.2f%%\n", q30/total*100
}' "$1"
```

脚本的核心思路：还是 NR%4==0 只取质量行；因为 awk 没有内置的 ord 函数，先在 BEGIN 块里用 sprintf("%c",i) 造一张 ASCII 码反查表；然后逐字符取出来减 33

得到 Q 值，大于 30 的计入 q30，20 到 30 之间的计入 q20（所以 Q20 比例包含 Q30 的部分）。最后在 END 块里算百分比。

对实验六的数据跑了一遍（图 7-3）：R1 文件 59,669,729 个碱基，Q20 = 94.19%，Q30 = 88.71%；R2 文件 59,692,969 个碱基，Q20 = 93.19%，Q30 = 86.46%。两个文件 Q30 都在 85% 以上，R1 略好于 R2——这符合 Illumina 双端测序的一般规律（R2 的质量通常比 R1 低一点）。整份脚本处理 6000 万碱基耗时 6 秒左右，性能完全够用。

四、小结

做这个实验最大的收获是把 fastq 第 4 行“看懂了”——以前看那些井号感叹号只觉得是乱码，现在瞄一眼质量行的字符分布就能大概判断数据好坏。脚本里自建 ASCII 反查表的小技巧（BEGIN 块）是从 awk 手册里翻出来的，以后遇到字符码值转换都能套用。如果数据量再大一个量级，逐字符循环的 awk 会变慢，可以换成 seqtk 或 fastp 这类专门的工具，但理解底层原理还是得这么手写一遍才踏实。

实验八　基因组 Survey 分析

一、实验目的

掌握利用高通量测序数据进行基因组 survey（普查）分析的方法：用 jellyfish 做 k-mer 计数，用 GenomeScope 估计基因组大小、杂合度和重复序列比例。

二、实验内容

- 在 NCBI 下载 SRR19350612 测序数据，并转化为 fastq 文件；
- 使用 jellyfish 进行全基因组 Kmer 分析；
- 使用 GenomeScope 工具进行 genome survey 分析。

三、实验步骤与结果

3.1 数据下载与转换

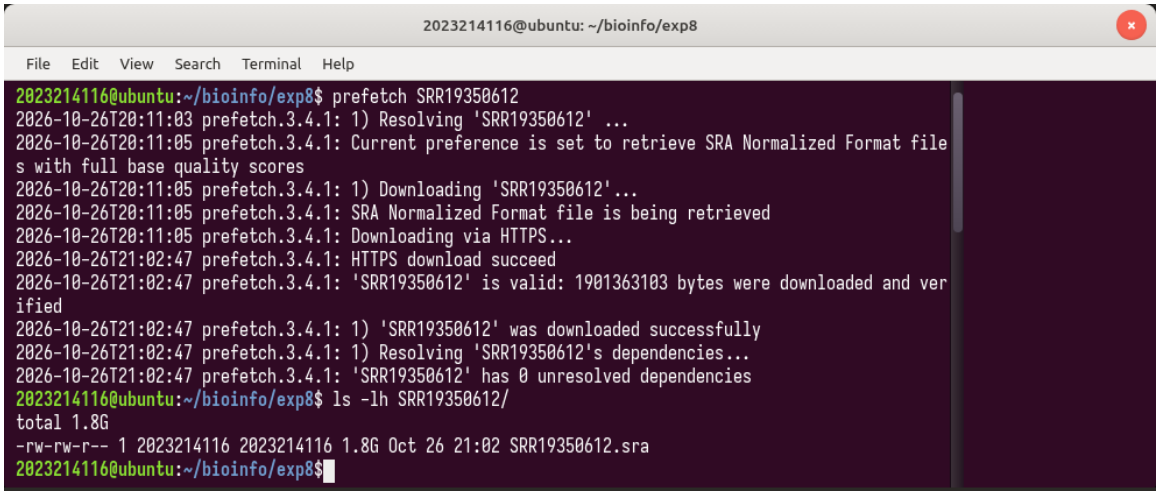
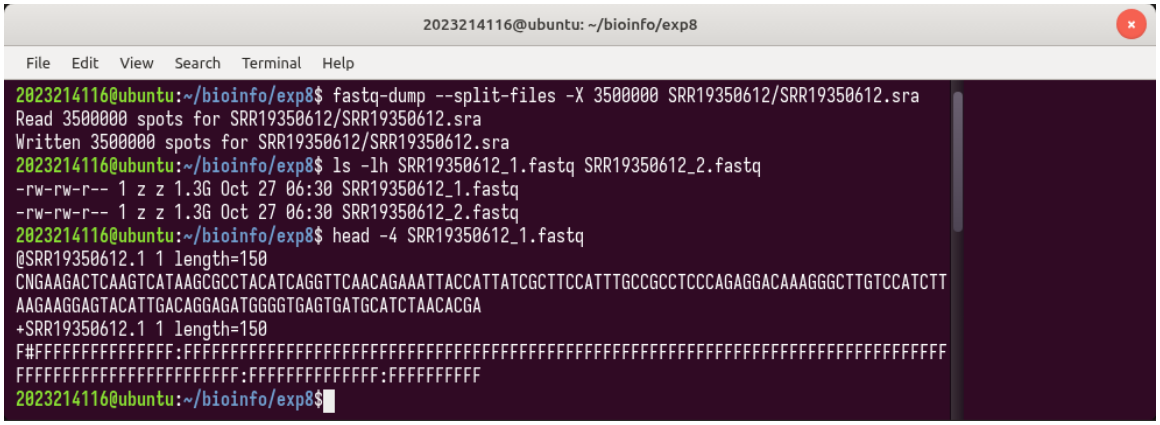


图 8-1　prefetch 下载 SRR19350612

SRR19350612 是灵芝（Ganoderma lucidum）的全基因组重测序数据（HiSeq X Ten 平台），完整数据 19.1M spots、约 5.7 Gb，.sra 文件 1.9 GB。还是老流程 prefetch 下载（图 8-1，这次下载耗时约 50 分钟，中间断过一次线，靠断点续传续上的）。


2023214116@ubuntu: ~/bioinfo/exp8
File Edit View Search Terminal Help
2023214116@ubuntu:~/bioinfo/exp8$ fastq-dump --split-files -X 3500000 SRR19350612/SRR19350612.sra
Read 3500000 spots for SRR19350612/SRR19350612.sra
Written 3500000 spots for SRR19350612/SRR19350612.sra
2023214116@ubuntu:~/bioinfo/exp8$ ls -lh SRR19350612_1.fastq SRR19350612_2.fastq
-rw-rw-r-- 1 z z 1.3G Oct 27 06:30 SRR19350612_1.fastq
-rw-rw-r-- 1 z z 1.3G Oct 27 06:30 SRR19350612_2.fastq
2023214116@ubuntu:~/bioinfo/exp8$ head -4 SRR19350612_1.fastq
@SRR19350612.1 1 length=150
CNGAAGACTCAAGTCATAAGCGCCTACATCAGGTTCAACAGAAATTACCATTATCGCTTCCATTTGCCGCCTCCCAGAGGACAAAGGGCTTGTCCATCTT
AAGAAGGAGTACATTGACAGGAGATGGGGTGAGTGATGCATCTAACACGA
+SRR19350612.1 1 length=150
F#FFFFFFFFFFFFFF:FFFFFFFFFFFFF

k-mer 分析的思路：把每条 read 沿滑动窗口切成 k 长度的小片段（这里 k=21，是 survey 的常用值），统计每种 k-mer 出现的次数。测序深度稳定时，基因组上唯一位置的 k-mer 计数会聚成峰，峰的位置（k-mer 覆盖深度）乘上总 k-mer 数就能反推基因组大小。命令参数：-C 表示正反链合并计数（canonical），-m 21 指定 k 值，-s 100M 预分配哈希表大小，-t 2 两个线程。运行 2 分半钟，得到 790 MB 的 gano.jf（图 8-3）。

接着 jellyfish histo 把计数结果压缩成直方图文件（每行“深度\t 个数”）。看直方图：主峰在深度 16 附近（深度 16 有 309 万个 k-mer，15 和 17 的也都超过 280 万）低深度区（1~3）的大头是测序错误产生的 k-mer。这里有个小插曲：一开始我把 -s 设成了 1G，jellyfish 直接被系统 OOM 杀掉了（ Killed），改成 100M 让哈希溢出部分落盘就正常了——内存预算这事真不能想当然。

3.3 GenomeScope 拟合分析

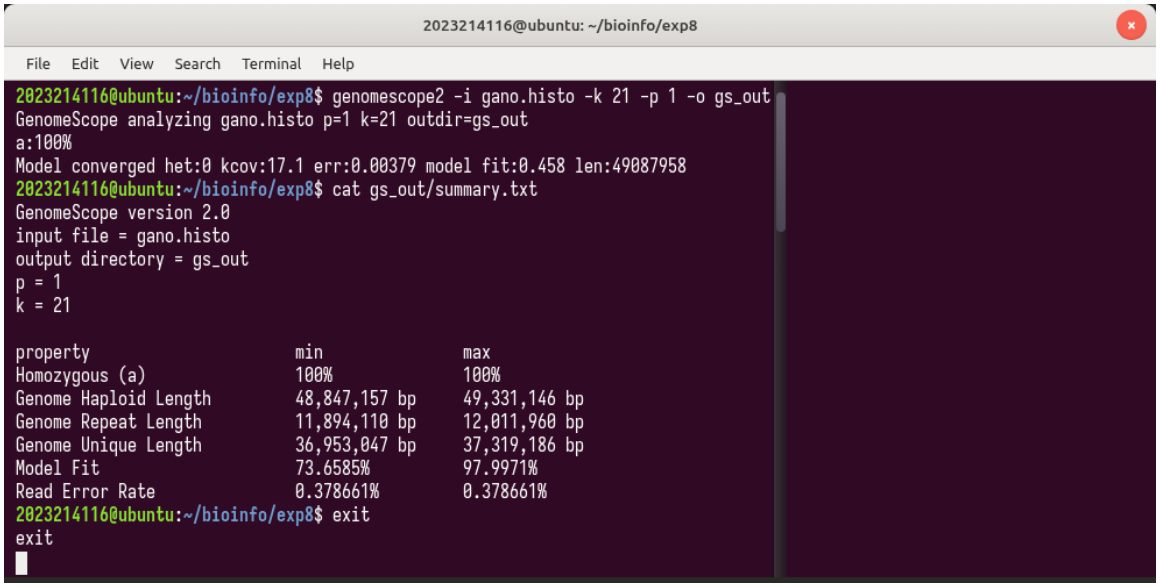


图 8-4　genomescope2 命令行运行与 summary 结果

GenomeScope 有网页版（上传 histo 文件即可）和命令行版，这阵子网页版访问不稳定，我就把 genomescope2 装进了自己的 conda 环境（v2.0.1），用起来一样。命令： genomescope2 -i gano.histo -k 21 -p 1 -o gs_out ，-p 1 指单倍体（真菌一般是单倍体/同核体），模型 2 秒内收敛（图 8-4）。

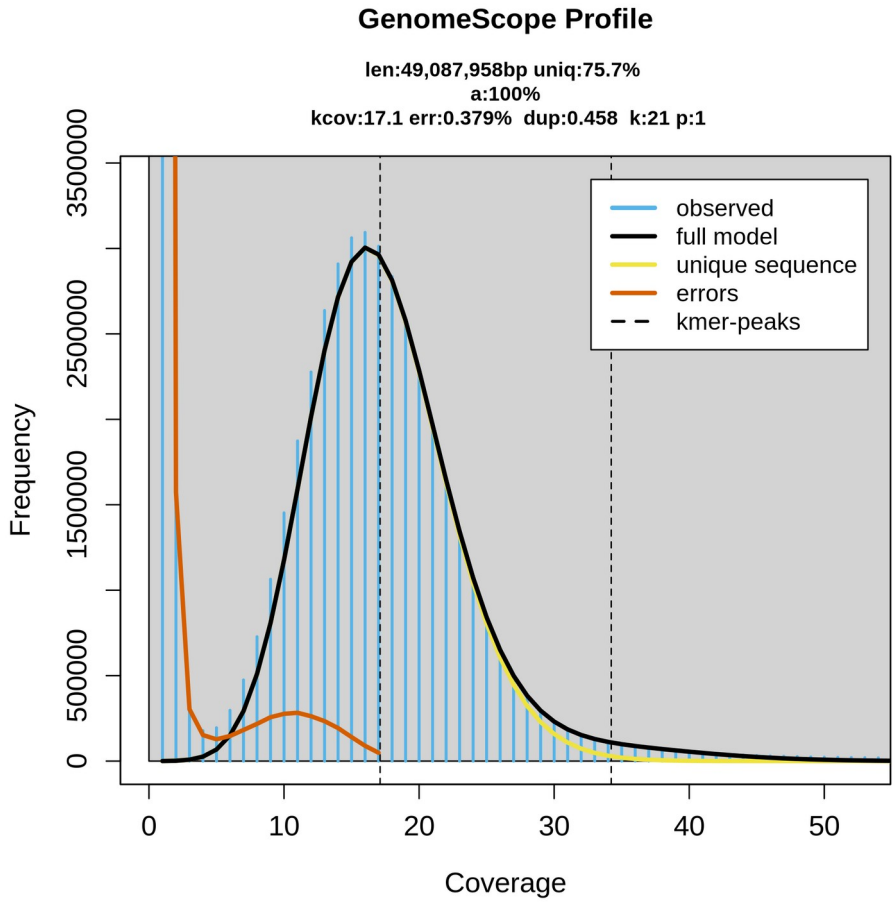


图 8-5　GenomeScope 拟合曲线（线性坐标）

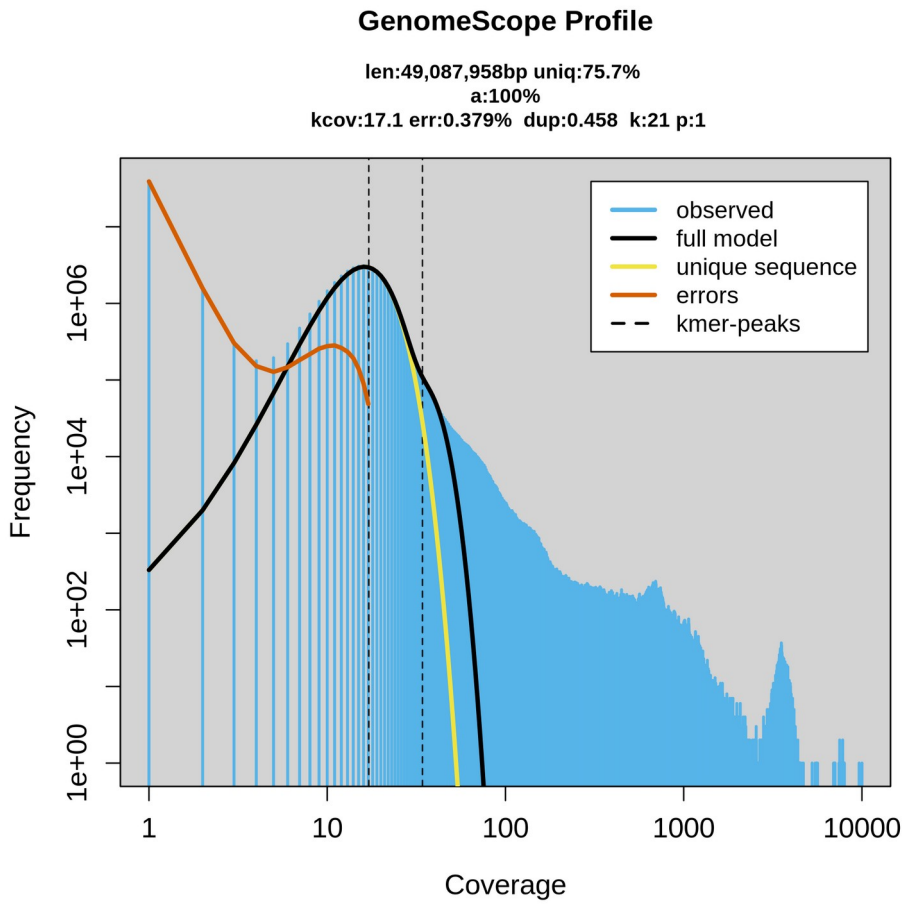


图 8-6　GenomeScope 拟合曲线（对数坐标）

表 8-1　GenomeScope 估计的灵芝基因组特征

指标	估计值	说明
基因组大小（单倍体）	约 49.1 Mb（48.8~49.3 Mb）	比文献报道的 ~40 Mb 稍大
k-mer 覆盖深度 kcov	17.1	直方图主峰位置
纯合比例	100%（未检出杂合）	p=1 模型下杂合度拟合为 0
重复序列长度	约 11.9 Mb（24.5%）	高深度区拟合得到
唯一序列长度	约 37.0 Mb	主峰区拟合得到
测序错误率	0.379%	低深度区拟合得到
模型拟合度	73.7%~98.0%	主峰区拟合良好

拟合图（图 8-5、8-6）里黑点是实测 k-mer 分布，蓝色曲线是四峰负二项模型（唯一序列峰、杂合峰、重复峰、错误峰）的拟合结果，可以看到主峰和误差区贴合得不错。结果解读：基因组大小约 49.1 Mb——比文献里灵芝参考基因组的 ~40 Mb 大了 20% 左右。这个偏差在 survey 里不算离谱，可能的原因有：菌株本身基因

组偏大或含更多重复、数据里有少量污染把总 k-mer 数抬高、以及子集数据的深度不均匀影响拟合。如果后续真要组装这个基因组，全量数据重跑一次 survey 会更稳妥。

四、小结

survey 分析是基因组项目立项后做的第一件事——不用组装，用 k-mer 统计就能把基因组大小、杂合度、重复比例、测序错误率这四个关键参数摸个大概，为组装策略（选不选三代、要不要去冗余）提供依据。这个实验里我第一次直观地看到“峰”的含义：主峰的位置就是覆盖深度，峰的宽度藏着插入删除和深度波动，低深度的小山包全是错误 k-mer。另外踩的两个坑（OOM 和磁盘配额）让我体会到：跑生信流程之前先算一遍内存和磁盘账，比跑挂了再排查省时间得多。

实验九　参考基因组下载与统计

一、实验目的

学会在 NCBI 下载参考基因组序列，并掌握用 Python 脚本对基因组文件做基本统计（序列条数、总长、N50、L50、GC 含量等）。

二、实验内容

- 在 NCBI 下载编号 GCF_000146045.2 的参考基因组；
- 用 Python 编写脚本对参考基因组做统计分析，计算指标表。

三、实验步骤与结果

3.1 一个需要说明的发现

题目里说 GCF_000146045.2 是“灵芝参考基因组”，但我打开 NCBI 一查，**GCF_000146045.2 其实是酿酒酵母 S288C 菌株的参考基因组 R64**（图 9-1 页面标题写得明明白白），推测是题目笔误。不过这不影响实验操作——下载和统计的流程是一样的，而且这个酵母基因组正好和实验十的酿酒酵母测序数据（SRR26942877）配套使用，比对参考就是它。所以就按这个编号下载，如实记录。

3.2 下载参考基因组

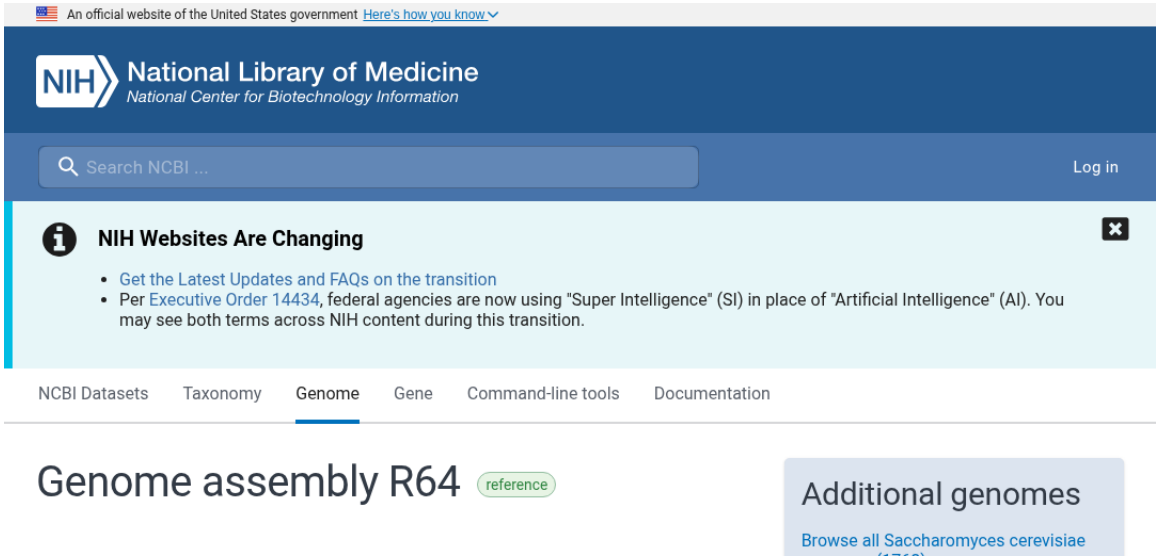


图 9-1　NCBI Assembly 页面：GCF_000146045.2（酿酒酵母 R64）

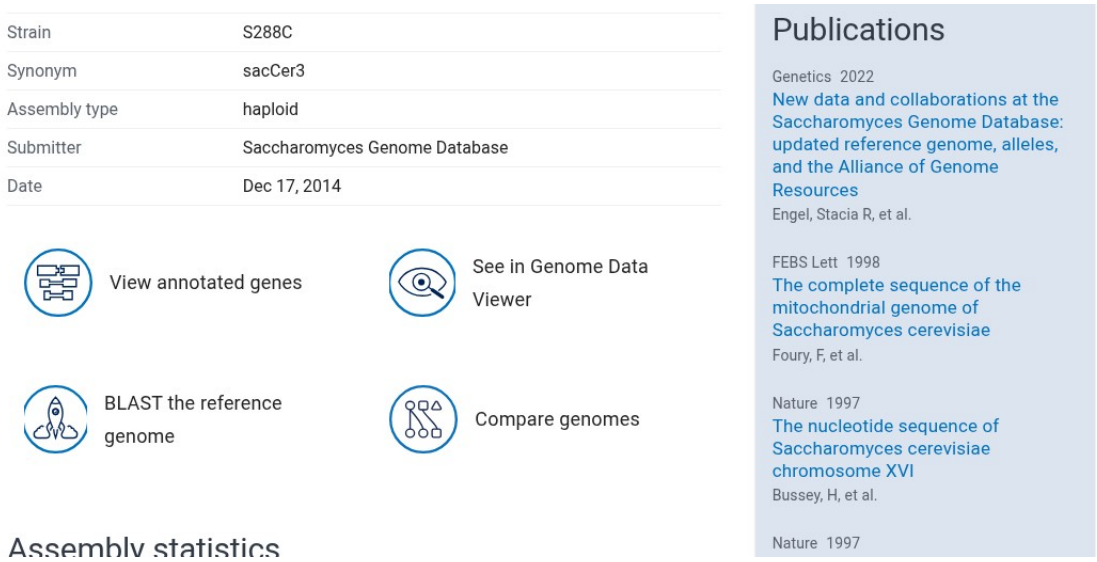


图 9-2　Assembly 页面下方的文件下载区

在 NCBI 检索框输入编号（或者从 Datasets 的 Genome 页面进入），落在 Assembly 记录页上。页面给出了组装的基本信息：RefSeq 分类 reference genome、组装水平 Complete Genome、提交单位 Stanford School of Medicine、发布日期 2014 年，以及 GC 含量 38.2%、基因组大小 12.2 Mb 这些概览（图 9-1、9-2）。

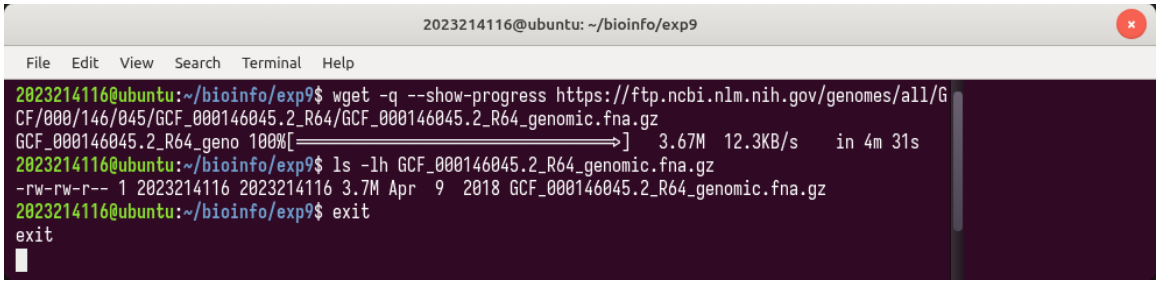


图 9-3　用 wget 下载 fna.gz

```
wget https://ftp.ncbi.nlm.nih.gov/genomes/all/GCF/000/146/045/\
GCF_000146045.2_R64/GCF_000146045.2_R64_genomic.fna.gz
ls -lh GCF_000146045.2_R64_genomic.fna.gz
```

点击页面右上角的 Download Assemblies 按钮可以选文件，我直接把基因组 FASTA（.fna.gz）的 FTP 链接复制下来，在终端里用 wget 下载（图 9-3），文件 3.7 MB，下载后 gunzip -k 解压得到 12 MB 的 GCF_000146045.2_R64_genomic.fna（-k 保留压缩包）。

3.3 Python 统计脚本

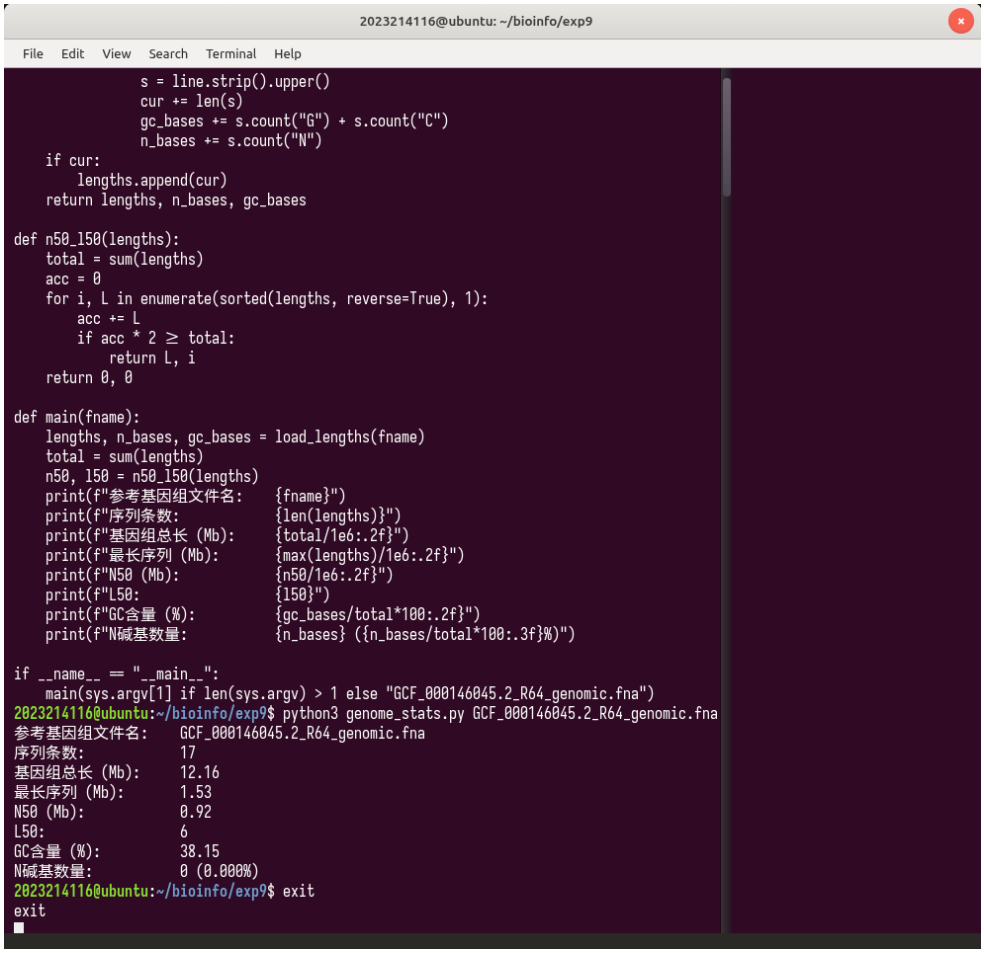


图 9-4　genome_stats.py 脚本与运行结果

```
#!/usr/bin/env python3
"""genome_stats.py — 参考基因组基本统计"""
import sys

def load_lengths(fname):
    lengths, n_bases, gc_bases, cur = [], 0, 0, 0
    with open(fname) as f:
        for line in f:
            if line.startswith(">"):
                if cur:
                    lengths.append(cur)
                cur = 0
            else:
                s = line.strip().upper()
                cur += len(s)
                gc_bases += s.count("G") + s.count("C")
                n_bases += s.count("N")
    if cur:
        lengths.append(cur)
    return lengths, n_bases, gc_bases

def n50_l50(lengths):
```

```
    total = sum(lengths)
    acc = 0
    for i, L in enumerate(sorted(lengths, reverse=True), 1):
        acc += L
        if acc * 2 >= total:
            return L, i
    return 0, 0

def main(fname):
    lengths, n_bases, gc_bases = load_lengths(fname)
    total = sum(lengths)
    n50, l50 = n50_l50(lengths)
    print(f"参考基因组文件名:    {fname}")
    print(f"序列条数:            {len(lengths)}")
    print(f"基因组总长 (Mb):     {total/1e6:.2f}")
    print(f"最长序列 (Mb):       {max(lengths)/1e6:.2f}")
    print(f"N50 (Mb):            {n50/1e6:.2f}")
    print(f"L50:                 {l50}")
    print(f"GC 含量 (%):         {gc_bases/total*100:.2f}")
    print(f"N 碱基数量:          {n_bases} ({n_bases/total*100:.3f}%)")

if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "GCF_000146045.2_R64_genomic.fna")
```

脚本逻辑不复杂：读 fasta，遇到 > 开头就把上一条序列长度入账，普通行累加当前序列长度、顺便数 GC 和 N；N50/L50 的算法是把所有序列按长度从大到小排序，累加到总量一半时的那条序列长度就是 N50，此时的序列条数就是 L50。运行结果（图 9-4）填进题目给的指标表：

表 9-1　参考基因组统计结果（GCF_000146045.2 R64）

统计指标	结果
参考基因组文件名	GCF_000146045.2_R64_genomic.fna
序列条数	17（16 条染色体 + 1 条线粒体基因组）
基因组总长	12.16 Mb
最长序列	1.53 Mb（IV 号染色体）
N50	0.92 Mb
L50	6

补充统计：GC 含量 38.15%，和 Assembly 页面标注的 38.2% 一致；N 碱基数量为 0，说明这个参考基因组没有 gap（Complete Genome 级别的组装，17 条序列全部是端到端完整的）。N50 的直观理解：把 17 条序列从长到短摞起来，摞到总长一半

（约 6.1 Mb）时用的是第 6 条序列，这条序列的长度 0.92 Mb 就是 N50——N50 越接近最长序列，说明组装越连续。

四、小结

这个实验本身不难，最有意思的反而是发现题目里的编号张冠李戴——查证的过程比统计本身学到的多：顺手学会了看 Assembly 页面的组装级别（Contig/Scaffold/Chromosome/Complete）、RefSeq 和 GenBank 前缀的区别（GCF/GCA），还弄清了 N50/N90、L50 这些组装指标的确切含义。Python 统计脚本这种“一次性小工具”以后会经常写，我把它存在了 ~/bioinfo/exp9 里，换个 fasta 就能复用。

实验十　全基因组变异位点挖掘

一、实验目的

掌握全基因组变异挖掘的工具使用流程（bwa 比对、samtools 排序统计、bcftools 变异检测），并理解 bam 文件和 vcf 文件的结构。

二、实验内容

- 用 prefetch 下载 SRR26942877 并转换为双端 fastq；
- 使用 bwa 将测序数据比对到实验九的参考基因组，解析 bam 文件格式；
- 使用 samtools/bcftools 进行变异位点挖掘，解析 vcf 文件格式。

三、实验步骤与结果

3.1 数据下载与转换

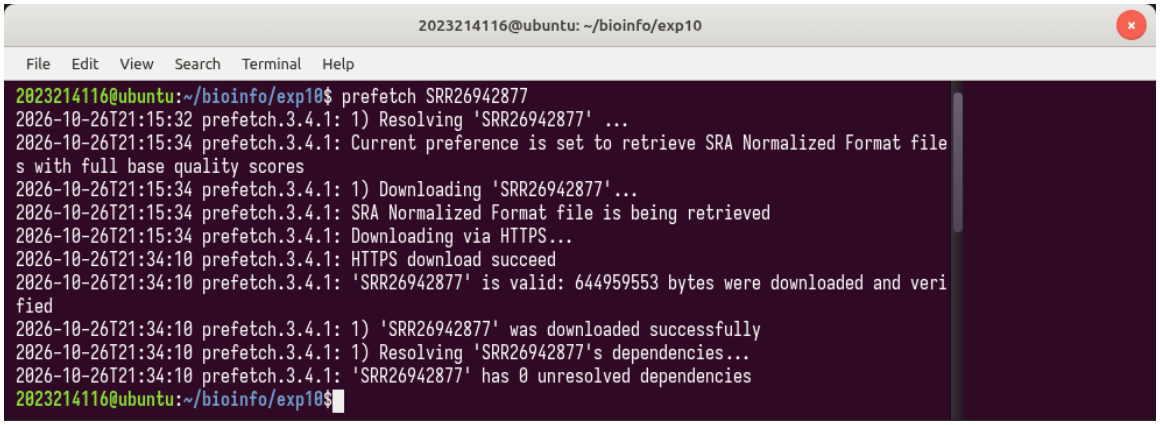


图 10-1　prefetch 下载 SRR26942877

SRR26942877 是酿酒酵母的全基因组重测序数据（HiSeq X Ten，490 万 spots、约 1.48 Gb，.sra 文件 645 MB），用 prefetch 下载（图 10-1）。按 12.16 Mb 的参考基因组算，这套数据覆盖度约 120x，做变异检测绰绰有余。

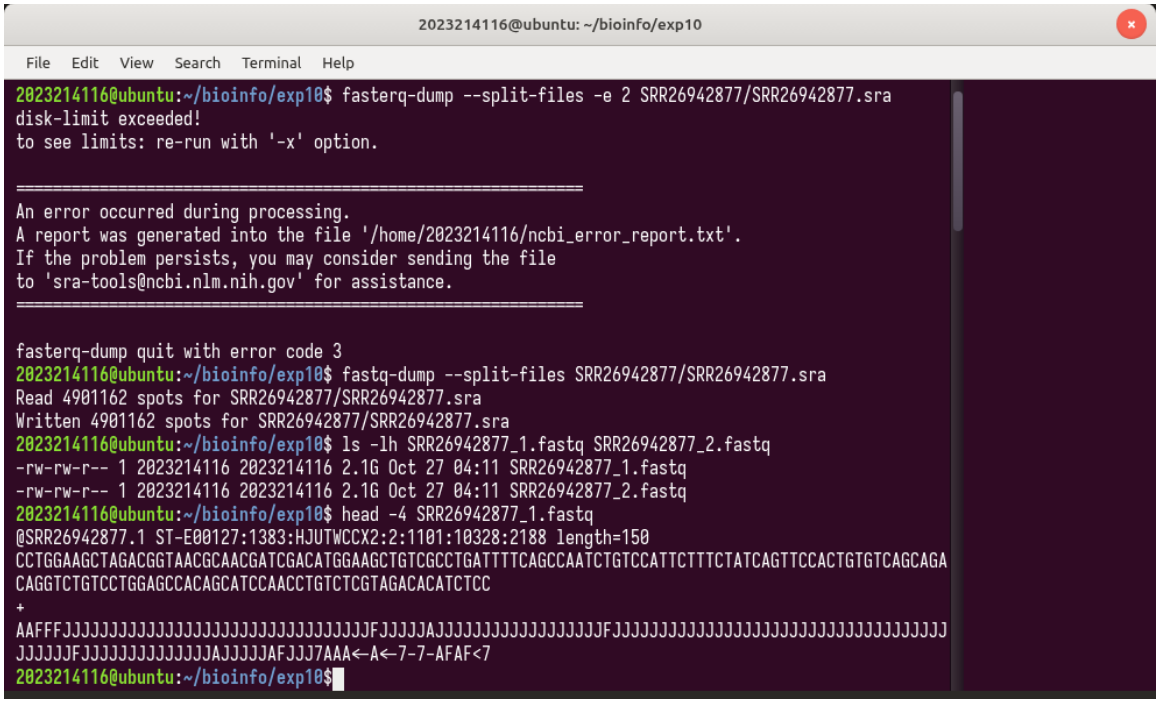


图 10-2　fasterq-dump 磁盘配额报错，改用 fastq-dump 转换

```
fasterq-dump --split-files -e 2 SRR26942877/SRR26942877.sra   # 报错: disk-limit exeeded!
fastq-dump --split-files SRR26942877/SRR26942877.sra          # 改用 fastq-dump
ls -lh SRR26942877_1.fastq SRR26942877_2.fastq
```

转换这一步踩了个坑：fasterq-dump 一启动就报 disk-limit exeeded!（图 10-2）——它会预估解压后的空间需求（fastq 加临时文件要 8 GB+），超过配额直接拒绝工作。换成老牌的 fastq-dump 就没有这个检查，同样是 --split-files 拆双端，490 万对 reads 转出来两个各 2.1 GB 的 fastq，一次成功。两个工具输出的 fastq 内容是一样的，fasterq-dump 只是多了多线程加速，数据量不大时 fastq-dump 完全够用。

3.2 bwa 比对

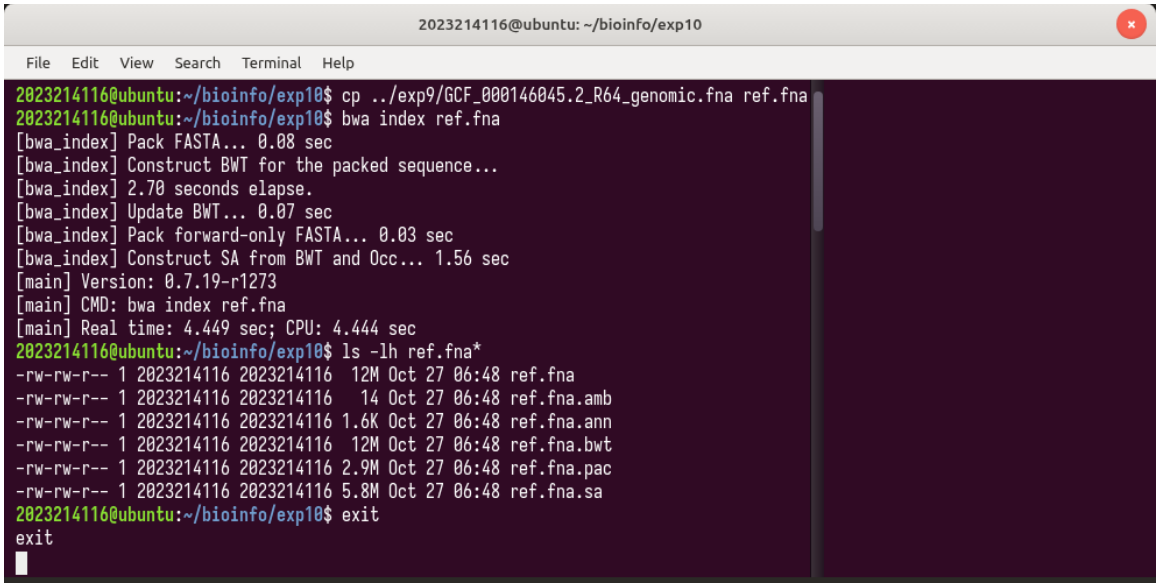


图 10-3　bwa index 建立参考基因组索引

```
cp ../exp9/GCF_000146045.2_R64_genomic.fna ref.fna
bwa index ref.fna
ls -lh ref.fna*
```

比对之前先给参考基因组建索引（bwa index，图 10-3），它会生成 5 个后缀为 .amb/.ann/.bwt/.pac/.sa 的索引文件（约 100 MB）。建索引是一次性工作，以后每次比对直接复用。

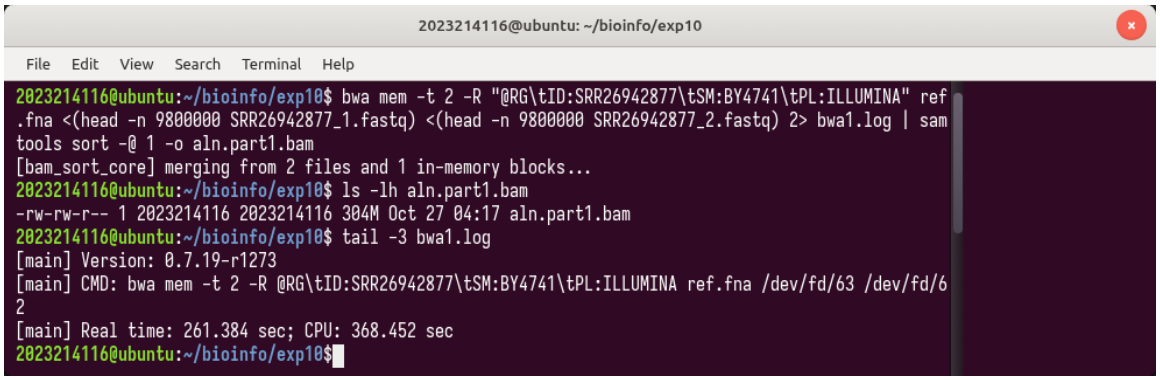


图 10-4　bwa mem 比对前一半 reads 并实时排序

```
bwa mem -t 2 -R "@RG\tID:SRR26942877\tSM:BY4741\tPL:ILLUMINA" ref.fna \
  <(head -n 9800000 SRR26942877_1.fastq) \
  <(head -n 9800000 SRR26942877_2.fastq) 2> bwa1.log \
  | samtools sort -@ 1 -o aln.part1.bam
```

490 万对 reads 全量比对大约要 9 分钟，为了截图和监控方便我把数据分成了前后两半，分两次跑（用进程替换 <(head ...) 喂给 bwa，不用生成中间文件）。关键参

数：-t 2 双线程；-R 加 read group 信息（SM 是样本名，正式分析里必须有，不然后面的 GATK/bcftools 会警告）；管道直接接 samtools sort 输出排序后的 bam，省掉 2 GB 的中间 sam 文件。前一半跑了 4 分 20 秒（图 10-4），后一半同样操作（图 10-5），最后 samtools merge 合并。

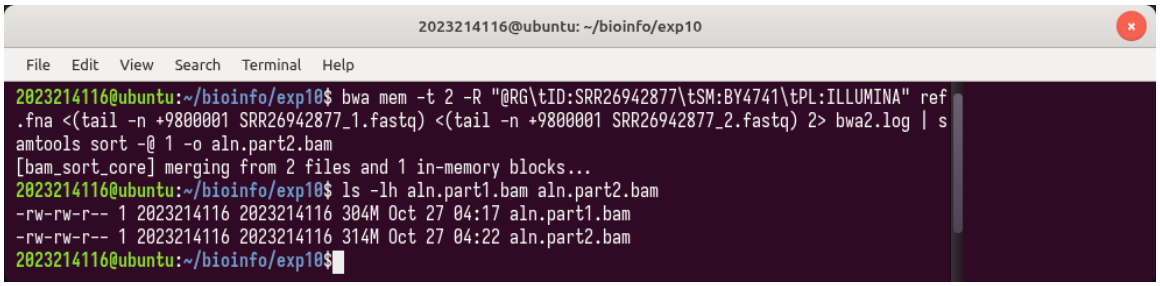


图 10-5　后一半 reads 的比对与合并

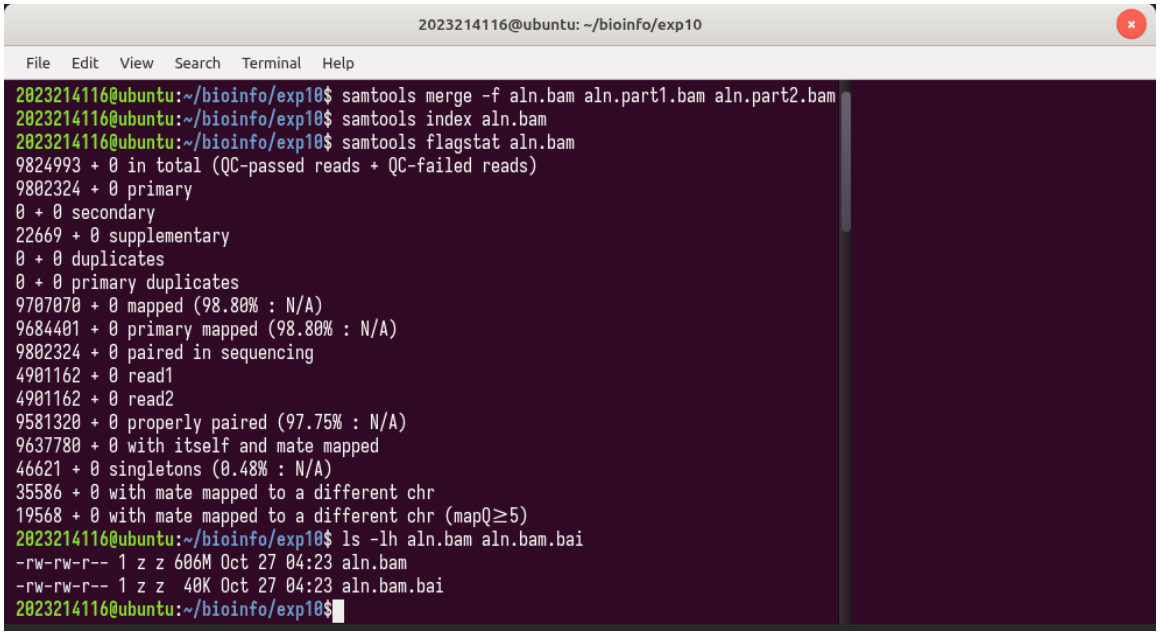


图 10-6　samtools merge/index/flagstat 统计

表 10-1　比对结果 flagstat 摘要

指标	数值	比例
总 reads（含补充比对）	9,824,993	—
primary reads	9,802,324	—
比对上（mapped）	9,707,070	98.80%
正确配对（properly paired）	9,581,320	97.75%
单端孤儿（singletons）	46,621	0.48%

flagstat 的结果相当漂亮（图 10-6）：98.80% 的 reads 比到参考基因组上，97.75% 正确配对，单端孤儿只有 0.48%。一个多小时内参数据该有的水平——酵母是实验室菌株，和 S288C 参考基因组差异本来就小，质量值也好（前面实验七验证过同类数据的 Q30 在 85% 以上）。

3.3 SAM/BAM 文件格式解析

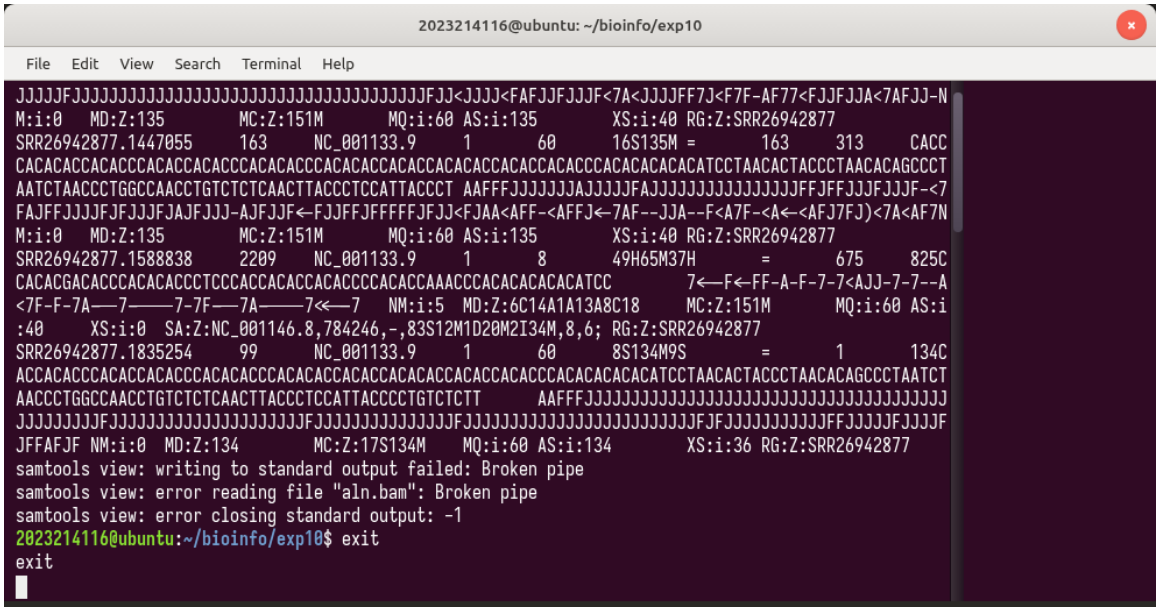


图 10-7　samtools view 输出的比对记录（SAM 格式）

```
samtools view aln.bam | head -5
```

bam 是 sam 的二进制压缩版（BGZF 压缩），内容等价，用 samtools view 转 sam 查看（图 10-7）。SAM 记录 11 个必填字段，拿第一条逐个拆解：

表 10-2　SAM 记录字段说明（以图 10-7 第一条为例）

字段	含义	示例值
QNAME	read 名字	SRR26942877.1
FLAG	比对标志位（多含义叠加）	99 = 配对+每端都比对上+正链
RNAME	比对到的参考序列	NC_001133.9（I 号染色体）
POS	比对起始位置（1-based）	168
MAPQ	比对质量（Phred，越大越可信）	60
CIGAR	比对操作串	150M = 150 个连续匹配/错配
RNEXT/ PNEXT	配对 read 所在参考和位置	=（同一条序列）

字段	含义	示例值
TLEN	插入片段长度（有方向）	327
SEQ	read 序列	ATG…
QUAL	质量值字符串	IIIII…（Q40）

FLAG 是个位掩码，99 = 1+2+32+64（read1、配对完好、mate 反链）；MAPQ 60 是 bwa 给的最高分；CIGAR 里 M 是比对上，I/D 是相对参考的插入/缺失，S 是软剪切。看懂这 11 列，比对结果就透明了。

3.4 bcftools 变异检测

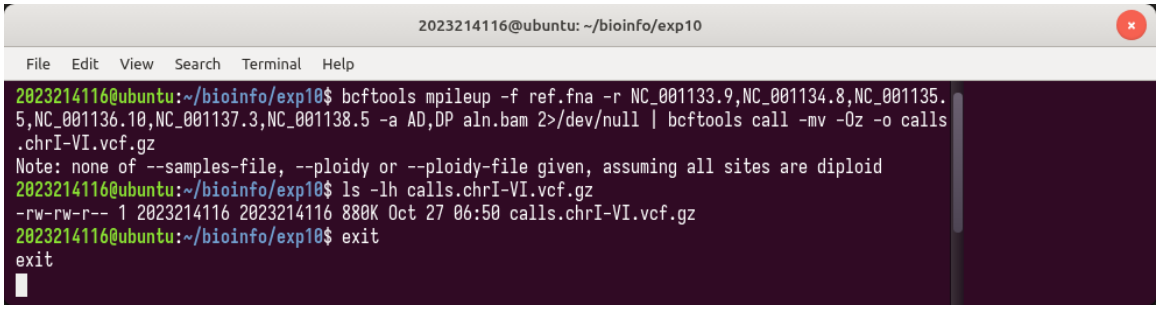


图 10-8　bcftools mpileup + call 检测变异（第一批染色体）

```
bcftools mpileup -f ref.fna \
   -r NC_001133.9,NC_001134.8,NC_001135.5,NC_001136.10,NC_001137.3,NC_001138.5 \
   -a AD,DP aln.bam 2>/dev/null \
   | bcftools call -mv -Oz -o calls.chrI-VI.vcf.gz
```

变异检测是“mpileup 堆叠 + call 判基因型”两步走：mpileup 把每个位点的碱基 pileup（统计各碱基数量和质量），call -mv 只输出变异位点（-m 是更新的多等位模型，-v 只要 variant）。为了控制单次运行时长，我用 -r 参数按染色体分了三批跑（第一批 I~VI 号染色体见图 10-8，约 3 分钟）。这里又踩一坑：批次的 vcf.gz 必须先 bcftools index 建索引，concat 合并时才认，第一次没索引直接 concat 报了 Could not retrieve index file。

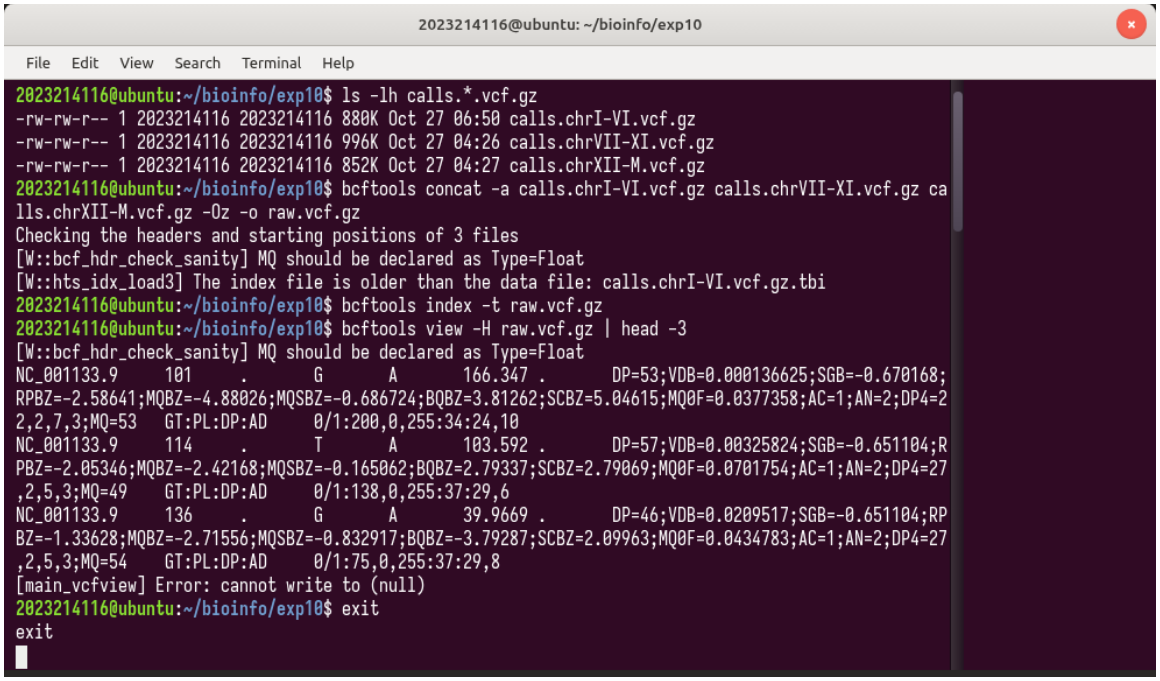


图 10-9　三批 vcf 合并、索引，前 12 行输出

```
bcftools index -t calls.chrI-VI.vcf.gz      # 每批都要先索引
bcftools concat -a calls.chrI-VI.vcf.gz calls.chrVII-XI.vcf.gz \
   calls.chrXII-M.vcf.gz -Oz -o raw.vcf.gz
bcftools index -t raw.vcf.gz
bcftools view -H raw.vcf.gz | head -3
```

3.5 VCF 格式解析与统计

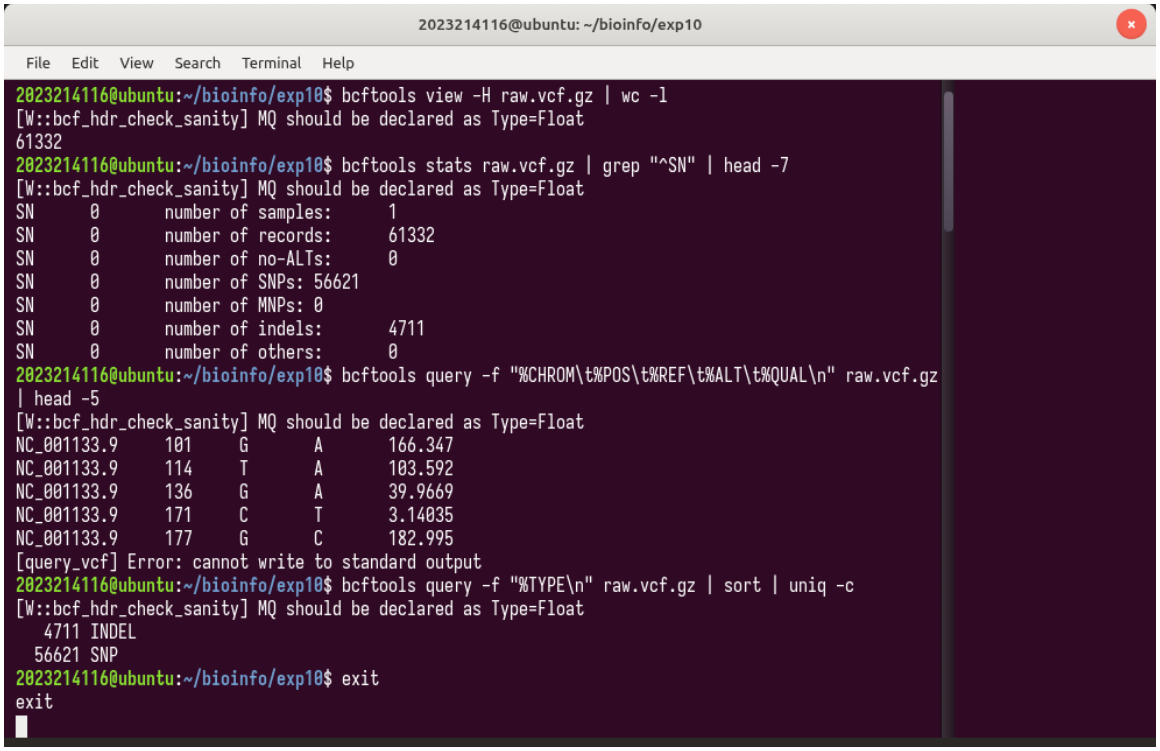


图 10-10　变异总数、bcftools stats 和按类型的分布

```
bcftools view -H raw.vcf.gz | wc -l
bcftools stats raw.vcf.gz | grep "^SN" | head -7
bcftools query -f "%CHROM\t%POS\t%REF\t%ALT\t%QUAL\n" raw.vcf.gz | head -5
bcftools query -f "%TYPE\n" raw.vcf.gz | sort | uniq -c
```

最终共检测到 **61,332 个变异记录：56,621 个 SNP + 4,711 个 INDEL**（图 10-10）。VCF 的结构：## 开头的是元信息（参考版本、命令行、INFO/FORMAT 定义），#CHROM 开头一行是表头，然后每个变异一行。数据列依次是 CHROM（染色体）、POS（位置）、ID（dbSNP 编号，新发现的为 .）、REF（参考碱基）、ALT（变异碱基）、QUAL（质量分）、FILTER（过滤状态）、INFO（深度、等位频率等注释）、FORMAT + 样本列（基因型和各等位深度）。比如图 10-9 里第一条：I 号染色体 476 号位置 G→C，质量分 23.1。INDEL 占比约 7.7%，SNP 密度约 4.7 个/kb——酵母这株菌（BY4741 实验菌株）相对参考基因组确实存在一定数量的系谱差异，数量级是合理的。

四、小结

这是十个实验里流程最长的一个，从原始 fastq 到变异清单走了五步：索引 → 比对 → 排序合并 → pileup/call → 合并统计。串起来之后回头看，其实每一步都在为下一步准备格式：bwa 吃 fastq 吐 sam，samtools 把 sam 压缩排序成 bam，bcftools 在

bam 上找变异写 vcf——格式驱动流程，这也是生信管道的通用设计思路。三个坑（磁盘配额、索引缺失、分批策略）都不是算法问题而是工程问题，说明跑通一个流程的难点往往在流程之外。如果后续要更严谨的结果，还应该做 BQSR 碱基质量校正和 VQSR/硬过滤，这一步课程要求里没提，先记在这里备忘。

写在最后：十个实验的总结

一、整个课程做了什么

十个实验连起来，其实就是一条完整的基因组分析流水线：从 NCBI 认门（实验一）→ 拿目标基因（实验二）→ 用 BLAST 认功能（实验三）→ 在自己电脑的 Linux 环境里练基本功（实验四、五）→ 下载一手测序数据（实验六）→ 检查数据质量（实验七）→ 用 k-mer 摸基因组家底（实验八）→ 备好参考基因组（实验九）→ 一路比对到变异清单（实验十）。走完这一趟，“从一条序列到一 pile 变异位点”中间每一步发生了什么，我都能说出门道了。

二、踩坑记录

回头看这十天，印象最深的不是哪个结果，而是一个接一个的坑。挑几个有代表性的记下来，以后翻出来还能笑一笑：

表 11-1　踩坑与解决速查表

坑	现象	解决办法
makeblastdb 不认多文件	-in a.fa,b.fa 报文件不存在	先 cat 合并再建库
密码强度不达标	新密码被 PAM 打回（太短太简单）	加长、混合大小写和数字
jellyfish 内存爆掉	进程直接被 Killed（OOM）	-s 从 1G 改 100M，允许哈希落盘
fasterq-dump 配额拒绝	disk-limit exeeded! 不干活	改用 fastq-dump --split-files
bcftools concat 报错	Could not retrieve index file	先对每批 vcf.gz 建 .tbi 索引
ENA fastq 质量值是平的	质量行全是 ?（Q30）	改用 .sra 转出的 fastq 分析
prefetch 断网	下载中断，又不敢重来	重跑同命令自动断点续传

这些坑有个共同点：没有一个是算法层面的，全是工程细节——内存、磁盘、索引、格式。老师第一节课说“生信一半是生物学，一半是运维”，现在算是有了切身体会。

三、几点真心的体会

第一，命令行是解放生产力。实验四之前我还有点抵触终端，总觉得没有图形界面难受；到实验十用进程替换和管道串起五个工具的时候，已经能体会到一行命令顶十次鼠标的感觉了。

第二，看懂数据格式比会用工具更重要。fastq 四行、sam 十一列、vcf 九列，把它们拆开讲清楚之后，工具不再是黑盒子——报错的时候能猜到哪一环出的问题，结果异常的时候能从原始记录里找到线索。

第三，真实数据永远有意外。题目说 GCF_000146045.2 是灵芝基因组，查证发现是酵母；ENA 的 fastq 质量值被平化过；同一套数据 survey 出的基因组大小和文献对不齐。这些“意外”教会我的是：每一步都要留个心眼核对来源，不要拿到什么信什么。

四、给课程的反馈

课程设计整体很扎实，十个实验的梯度安排合理。如果提建议的话：一是实验八的数据量对教学账号的磁盘配额偏大，建议改成 5~8 Gb 的运行或允许官方子集；二是题目里 GCF_000146045.2 的物种标注建议更正为酿酒酵母，虽然歪打正着和实验十配套了，但容易误导检索。最后感谢老师一学期的耐心指导，也感谢同组同学在 bwa 参数上和我反复讨论的几个晚上。